

Generative KI für 3D-Medizinbildgebung mittels Diffusion Models

Generative AI for 3D Medical Imaging using Diffusion Models

Studienarbeit

vorgelegt von

Sven Beller

2026

Matrikelnummer, Kurs

5102447, TINF23

Betreuer

Malte Tölle

Abstract

This is the abstract in English.

Zusammenfassung

Dies ist die Zusammenfassung auf Deutsch.

Inhaltsverzeichnis

Abbildungsverzeichnis	IX
Tabellenverzeichnis	XI
Nomenklatur	XII
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	1
1.3 Vorgehensweise	1
2 Medizinische Bildgebung	2
2.1 Bildgebende Verfahren	2
2.1.1 Magnetresonanztomographie	2
2.1.2 Computertomographie	3
2.2 Datenformat	3
3 Künstliche Intelligenz	4
3.1 Definition Künstliche Intelligenz	4
3.2 Symbolische und subsymbolische KI	4
3.3 Maschinelles Lernen	5
3.3.1 Supervised Learning	5
3.3.2 Unsupervised Learning	6
3.3.3 Bestärkendes Lernen	6
3.4 Artificial Neural Networks	7
3.4.1 Netzwerkarchitekturen	7
3.4.2 Perceptron	8
3.5 Unterscheidung zwischen Diskriminative und Generative Modelle	9
3.5.1 Diskriminative Modelle	9
3.5.2 Generative Modelle	9
3.6 Arten der Generierung	10

3.6.1	Unconditional	10
3.6.2	Conditional	10
3.7	Einordnung der Diffusionsmodelle	11
4	Stand der Forschung	12
4.1	GAN-basierte Ansätze	12
4.2	VAE-basierte Ansätze	12
4.3	Diffusionsbasierte Ansätze	13
5	Deep Generative Model	14
5.1	Generative Adversarial Networks	14
5.2	Variational Autoencoders	14
5.3	Diffusion Model	15
5.3.1	Markov-Ketten	15
5.3.2	Noise-Schedule	16
5.3.3	Vorwärtsprozess	16
5.3.4	Rückwärtsprozess	17
5.3.5	Sampling	18
5.4	Grundlegende Architekturen	21
5.4.1	Convolutional Neural Network	21
5.4.2	U-Net	22
6	Datenqualität und ethische Aspekte	25
6.1	Definition synthetischer Daten	25
6.2	Ethische Aspekte synthetischer medizinischer Daten	26
6.3	Risiken und Grenzen synthetischer Daten	26
6.4	Qualitätskriterien für synthetische medizinische Bilddaten	27
6.4.1	Visuelle Bewertungskriterien	27
6.4.2	Mathematische Bewertungsmetriken	27
7	Implementierung und experimenteller Aufbau	32
7.1	Projektmethodik	32
7.2	Entwicklungsumgebung und Hardware	32
7.3	Dataset und Datenvorverarbeitung	33
7.4	Modellarchitektur	34

7.5	Baseline-Konfiguration	35
7.6	Trainingsverfahren und Hyperparameter	36
7.7	Ablationsstudien	36
7.8	Aufgezeichnete Werte während des Trainings	37
7.9	Conditional-Modelle	37
7.10	Generierungs-Pipeline	38
7.11	Sample-Generierung	39
7.12	Testdurchlauf	39
8	Ergebnis	41
8.1	Trainingsverlauf	41
8.2	Quantitative Auswertung der unconditional Modelle	43
8.3	Erhöhte Auflösung (48 ³)	44
8.4	Generierungs-Pipeline	45
8.5	Qualitative Ergebnisse	45
8.6	Sample-Generierung	50
9	Auswertung	52
10	Schlussfolgerung	53
10.1	Fazit	53
10.2	Ausblick	53
	Literaturverzeichnis	XIV
A	Anhang A	XXIV

Abbildungsverzeichnis

2.1	Darstellung eines beispielhaften Neuroimaging Informatics Technology Initiative (NIfTI)-Header [6, S. 76]	3
3.1	Schematische Darstellung eines Perceptrons	8
5.1	Markov-Kette erster Ordnung [15, S. 609]	16
5.2	Darstellung der Vorwärts- und Rückwärtsprozesse	17
5.3	Convolutional [50, S. 6]	21
5.4	CNN	22
5.5	U-Net Architektur [54, S. 2]	23
6.1	Darstellung der Fréchet Inception Distance (FID)-Berechnung [71, S. 130] . .	28
6.2	Darstellung der Structural Similarity Index Measure (SSIM)-Berechnung [74, S. 5]	29
8.1	Trainings- (durchgezogen) und Validierungs-Loss (gestrichelt) über 300 Epochen für alle acht Modelle.	42
8.2	Trainingsdauer der acht Modelle in GPU-Stunden auf einer NVIDIA GeForce GTX 1080 Ti.	42
8.3	Verteilung der paarweisen Metriken über 1024 Samples für die fünf 32^3 -Modelle, dargestellt jeweils für den FID-besten Sampler je Modell.	44
8.4	Je acht zufällig ausgewählte Samples der fünf 32^3 -Modelle (axialer Mittelschnitt). Zeilen von oben nach unten: Baseline, Data Augmentation, Linearer Schedule, Ohne Self-Attention, Noise-Prediction.	46
8.5	Vergleich der Baseline in 32^3 (obere Reihe) und 48^3 (untere Reihe), je sechs zufällig ausgewählte Samples mit Diffusion Denoising Probabilistic Model (DDPM)-Sampler.	47
8.6	Identisches Startrauschen für die Baseline 32^3 , dekodiert durch DDPM (250 Schritte), Diffusion Denoising Implicit Model (DDIM) (50), DPM-Solver++ (25) und Unified Predictor-Corrector (UniPC) (10).	48
8.7	Sechs Pipeline-Ausgaben mit UniPC: generierte Tumormaske (links), zugehöriges generiertes T1N-Volumen (Mitte) und Überlagerung beider (rechts). .	49
8.8	Denoising-Trajektorie eines Samples der Baseline 32^3 mit DDIM, dargestellt an acht äquidistanten Zwischenschritten von $t = T$ (links) bis $t = 0$ (rechts). .	50

Tabellenverzeichnis

8.1	Zusammenfassung des Trainingsverlaufs aller Modelle über 300 Epochen. <i>Beste Epoche</i> bezeichnet die Epoche mit dem niedrigsten Validierungs-Loss.	41
8.2	Quantitative Auswertung der fünf 32 ³ -Modelle über 1024 generierte Samples je Sampler. Niedrigere Werte sind besser bei FID und LPIPS, höhere bei SSIM und PSNR. Beste Werte je Modell sind fett markiert.	43
8.3	Quantitative Auswertung der 48 ³ -Baseline über 1024 generierte Samples je Sampler. Beste Werte je Spalte sind fett markiert.	44
8.4	Quantitative Auswertung der Generierungs-Pipeline (Klasse → Maske → MRT) in 32 ³ über 1024 generierte Samples je Sampler. Beste Werte je Spalte sind fett markiert.	45
8.5	Generierungsleistung der vier Sampler für die unconditional Baseline auf einer NVIDIA GeForce GTX 1080 Ti, gemittelt über 1024 generierte Volumina je Konfiguration. Peak-VRAM laut <code>nvidia-smi</code> (Gesamtbelegung der GPU). Vollständige Werte für die übrigen Ablationsvarianten siehe Anhang.	51
8.6	Generierungsleistung der vier Sampler für die Generierungs-Pipeline (Klasse → Maske → MRT) in 32 ³ , gemittelt über 1024 Pipeline-Durchläufe. Die Gesamtdauer setzt sich aus den Laufzeiten der beiden conditional Modelle zusammen.	51

Nomenklatur

Abkürzungen

ANN	Artificial Neural Networks
BAPPS	Berkeley-Adobe Perceptual Patch Similarity
cGAN	conditional Generative Adversary Network
CNN	Convolutional Neural Network
CT	Computertomographie
DDIM	Diffusion Denoising Implicit Model
DDPM	Diffusion Denoising Probabilistic Model
DGM	Deep Generative Model
DSC	Dice Similarity Coefficient
ELBO	Evidence Lower Bound
FID	Fréchet Inception Distance
FNN	Feedforward Neural Network
FR	full-reference
GAN	Generative Adversary Network
HA-GAN	Hierarchical Amortized Generative Adversary Network
IQMs	Image Quality Metrics
IS	Inception score
KI	Künstliche Intelligenz
LDM	Latent Diffusion Model
LPIPS	Learned Perceptual Image Patch Similarity
LSGAN	Last Squares Generative Adversarial Networks
MDP	Markov Decision Process
ML	Machine Learning
MLP	Multi-Layer Perceptron
MRT	Magnetresonanztomographie
MSE	Mean squared Error
NIfTI	Neuroimaging Informatics Technology Initiative
NIH	National Institutes of Health
NLP	Natural Language Processing
NR	no-reference
ODE	Ordinary Differential Equation
PSNR	Peak Signal-to-Noise Ratio
ReLU	Rectified Linear Unit

RNN	Recurrent Neural Network
RR	reduced-reference
SSIM	Structural Similarity Index Measure
UniPC	Unified Predictor-Corrector
VAE	Variational Autoencoder Network
VQ-GAN	Vector Quantized Generative Adversarial Network
EMA	Exponential Moving Average
BraTS	Brain Tumor Segmentation

1 Einleitung

1.1 Problemstellung

1.2 Zielsetzung

1.3 Vorgehensweise

2 Medizinische Bildgebung

Die medizinische Bildgebung bildet die Datengrundlage für die in dieser Arbeit untersuchten Diffusionsmodelle. Dieses Kapitel stellt eine kurze Einleitung in die medizinische Bildgebung sowie das verwendete Datenformat vor.

2.1 Bildgebende Verfahren

In der medizinischen Bildgebung kommen je nach Fragestellung unterschiedliche Modalitäten zum Einsatz. Für die Aufnahme und volumetrische Darstellung eines Körpers und ihre Organe sind insbesondere die MRT und die CT etabliert. Beide liefern dreidimensionale Schnittbilder, unterscheiden sich jedoch grundlegend im physikalischen Wirkprinzip und im resultierenden Gewebekontrast.

2.1.1 Magnetresonanztomographie

Die *Magnetresonanztomographie* (MRT) basiert auf der Anregung von Wasserstoffkernen im Körpergewebe durch starke statische Magnetfelder und Radiofrequenzimpulse. Beim Zurückkehren in den Gleichgewichtszustand emittieren die Kerne ein messbares Signal, dessen räumliche Zuordnung über zusätzliche Gradientenfelder erfolgt [vgl. 1, S. 514 ff.]. Die MRT kommt ohne ionisierende Strahlung aus [vgl. 1, S. 536]. Ebenfalls hebt sie durch Variation der Aufnahmesequenz unterschiedliche Gewebeeigenschaften hervor [vgl. 1, S. 517]. In der klinischen Routine der Hirntumor bildgebung werden vier strukturelle Sequenzen kombiniert [vgl. 2, S. 3 ff.]:

- **T1-gewichtet (T1):** Native Referenzaufnahme, die zum Vergleich mit der kontrastverstärkten T1ce-Aufnahme dient.
- **T1 kontrastverstärkt (T1ce):** Aktiv wachsende Tumoranteile reichern den Kontrast stark an und erscheinen auf T1ce heller als auf der T1-Aufnahme.
- **T2-gewichtet (T2):** Tumorkomponenten erscheinen hyperintens.
- **FLAIR:** Edema stellen sich auf FLAIR-Aufnahmen hyperintens dar.

Diese vier Sequenzen bilden den klinischen Standard in der Hirntumor-Bildgebung [vgl. 2, S. 3].

2.1.2 Computertomographie

Die *Computertomographie* (CT) beruht auf der Messung der Röntgenstrahlungsschwächung beim Durchgang durch das Gewebe. Aus den so gewonnenen Projektionen wird rechnergestützt ein dreidimensionaler Datensatz rekonstruiert. Die CT ist in der radiologischen Routine das am weitesten verbreitete bildgebende Verfahren und insbesondere in der Traumadiagnostik etabliert, geht aber mit einer Strahlenbelastung einher [vgl. 3, S. 1 ff.].

2.2 Datenformat

Das NifTI-Format (.nii, .nii.gz) wurde in den frühen 2000er Jahren unter Förderung des National Institutes of Health (NIH) entwickelt und hat sich seitdem als Standardformat in der neuroimaging-basierten Forschung etabliert. Es ist speziell auf volumetrische 3D-Bilddaten ausgelegt und eignet sich daher besonders für Anwendungen wie MRT-Aufnahmen des Gehirns [vgl. 4, S. 203].

Als Rasterformat speichert NifTI die Bilddaten in Form von Voxeln, also Pixeln mit definierter Breite, Höhe und Tiefe [vgl. 5, S. 4]. Die ersten drei Dimensionen kodieren die räumlichen Koordinaten x , y und z , während die vierte Dimension den Zeitpunkt t abbildet und etwa bei funktionellen MRT-Aufnahmen genutzt wird [vgl. 6, S. 76]. Der Header einer NifTI-Datei umfasst im .nii-Format 352 Byte und enthält alle Metadaten, die für die Interpretation der Bilddaten benötigt werden [vgl. 6, S. 76]. Abb. 2.1 zeigt einen solchen Header beispielhaft. Über den Parameter ImageSize lässt sich etwa die Auflösung des Scans ablesen, in diesem Fall 256x156x256 Voxel. Im Gegensatz zu Formaten

Parameters	Value
Filename	group00001_T1w_CSF_probability.nii
Filesize	40894816
Description	'DAD210715'
ImageSize	[256 156 256]
xPixelDimensions	[1 1.3000 1]
Datatype	'single'
BitsPerPixel	32
SpaceUnits	'Millimeter'
TimeUnits	'Second'
MultiplicativeScaling	1

Abb. 2.1: Darstellung eines beispielhaften NifTI-Header [6, S. 76]

wie JPEG, die mit verlustbehafteter Kompression arbeiten und dadurch diagnostisch relevante Bildinformationen beeinträchtigen können, unterstützt NifTI über die Endung .nii.gz eine verlustfreie Kompression [vgl. 4, S. 203].

3 Künstliche Intelligenz

Dieses Kapitel führt kurz in die Grundbegriffe der KI ein und ordnet die in dieser Arbeit verwendeten Diffusionsmodelle in das Feld der KI ein. Es werden nur jene Konzepte vorgestellt, die für das Verständnis der weiteren Kapitel zu maschinellem Lernen und generativen Modellen notwendig sind.

3.1 Definition Künstliche Intelligenz

Der Begriff *Künstliche Intelligenz* (KI) ist nicht eindeutig definiert und wird je nach Disziplin unterschiedlich verwendet. Nahrstedt versteht KI primär als ein Forschungsgebiet und nicht als eine einzelne Methode [vgl. 7, S. 5]. In dieser Studienarbeit wird KI im technischen Sinne als ein Sammelbegriff für Methoden verstanden, mit denen Computer befähigt werden, Aufgaben zu erledigen, die üblicherweise menschliche Intelligenz erfordern [vgl. 8, S. 42] [vgl. 9, S. 3]. Im Kontext dieser Arbeit zählt hierzu das Erkennen, Interpretieren und Erzeugen komplexer Bilddaten in der medizinischen Bildgebung.

Zentrale Eigenschaften, die mit Intelligenz assoziiert werden, sind unter anderem die *Wahrnehmung* (Verarbeitung und Interpretation von Sensordaten), das *Schlussfolgern* (logisches und induktives Ableiten von Erkenntnissen, auch unter Unsicherheit und unvollständiger Information) sowie die *Anpassungsfähigkeit* und das *Lernen* aus Erfahrungen [vgl. 10, S. 1 f.].

In der Literatur wird zwischen *schwacher* bzw. *beschränkter* KI (*narrow AI*) und *starker* bzw. *allgemeiner* KI (*general AI*) unterschieden [vgl. 11, S. 1]. Eine schwache KI ist auf eine bestimmte Domäne beschränkt und kann erlernte Lösungsmuster nicht auf andere Bereiche übertragen [vgl. 12, S. 7]. Eine starke KI hingegen wäre in der Lage, beliebige intellektuelle Aufgaben auf menschlichem Niveau zu lösen. Dies existiert bislang jedoch ausschließlich als theoretisches Konzept. Würde sich eine solche starke KI selbstständig weiterentwickeln, käme es zur sogenannten *technologischen Singularität*. Einem hypothetischen Zeitpunkt, an dem die maschinelle Intelligenz die menschliche übertrifft und in eine *Superintelligenz* übergeht [vgl. 12, S. 7] [vgl. 13, S. 17 f.]. Die heutigen eingesetzten KI-Systeme sind nach dieser Klassifikation der schwachen KI zuzuordnen [vgl. 11, S. 1].

3.2 Symbolische und subsymbolische KI

Innerhalb der KI lassen sich zwei methodische Hauptströmungen unterscheiden, die sich grundlegend in der Art der Wissensrepräsentation und im Lernverfahren unterscheiden: die *symbolische* und die *subsymbolische* KI [9, S. 4 f.].

Die **symbolische KI** (auch *regelbasierte KI*) repräsentiert Wissen explizit durch Symbole und logische Regeln. Aus allgemeinen Regeln werden mittels Deduktion konkrete Schlussfolgerungen abgeleitet (Top-Down-Ansatz). Dies sind Expertensysteme, deren Wissensbasis manuell durch Fachexperten erstellt wird [vgl. 9, S. 4 f.].

Die **subsymbolische KI** (auch *konnektionistische KI*) lernt hingegen induktiv aus großen Datenmengen, ohne dass explizite Regeln vorgegeben werden. Wissen wird verteilt in den Gewichten künstlicher neuronaler Netze repräsentiert und ist nicht direkt interpretierbar (Bottom-Up-Ansatz) [9, S. 4 f.]. Dieser datengetriebene Ansatz bildet die Grundlage moderner Verfahren des maschinellen Lernens und des Deep Learning.

3.3 Maschinelles Lernen

Eine zentrale Methode der subsymbolischen KI ist das *Machine Learning* (ML). Hierbei wird einem Computer nicht jede Regel explizit vorgegeben, sondern er erlernt selbstständig aus Daten Muster und Zusammenhänge, die er anschließend auf neue, vergleichbare Situationen anwendet. Der Computer lernt dabei nicht im Sinne eines Auswendig lernens, sondern erkennt verallgemeinerbare Gemeinsamkeiten in den Trainingsbeispielen, die sich auf unbekannte Daten übertragen lassen [vgl. 8, S. 42]. Mitchell definiert dies mit:

„A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .“ [14, S. 2]

Bedeutet, ein Programm gilt also dann als lernend, wenn es seine Leistung bei einer Aufgabe T , gemessen durch eine Performanzmetrik P , durch Erfahrung E verbessert. Des weiteren, wird im maschinellen Lernen zwischen drei Hauptkategorien unterschieden.

3.3.1 Supervised Learning

Beim *supervised learning* (de: *überwachtes Lernen*) lernt das Modell aus gelabelten Trainingsdaten [vgl. 15, S. 3]. Jeder Datenpunkt besteht aus einem Eingabevektor $\mathbf{x}_i$ und einem zugehörigen Zielwert y_i , der die korrekte Antwort vorgibt. Die Trainingsmenge umfasst N solcher Paare $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ [vgl. 15, S. 137] [vgl. 16, S. 695 f.].

Ziel ist es, eine Funktion zu lernen, die einer beliebigen Eingabe $\mathbf{x}$ den passenden Zielwert zuordnet und sich auch auf neue, unbekannte Daten übertragen lässt [vgl. 16, S. 696]. Diese Funktion wird durch ein Modell f_{θ} mit lernbaren Parametern θ realisiert. Wie gut das Modell die Trainingsdaten trifft, wird über eine Verlustfunktion L gemessen, die den Fehler zwischen Vorhersage $f_{\theta}(\mathbf{x}_i)$ und tatsächlichem Wert y_i angibt. Beim Training werden die Parameter θ so angepasst, dass dieser Fehler über die gesamte Trainingsmenge minimiert wird [vgl. 17, S. 275 f.]:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(\mathbf{x}_i), y_i) \quad (3.1)$$

Aus probabilistischer Sicht entspricht das supervised learning dem Schätzen der bedingten Wahrscheinlichkeit $p(y | \mathbf{x})$ [vgl. 17, S. 105 f.] [vgl. 15, S. 137 f.]. Je nach Art des Zielwerts unterscheidet man zwischen *Klassifikation* (diskrete Klassen) und *Regression* (kontinuierliche Werte) [vgl. 15, S. 3] [vgl. 16, S. 696].

3.3.2 Unsupervised Learning

Beim *unsupervised learning* (de: *unüberwachtes Lernen*) erhält das Modell ausschließlich Eingabedaten ohne zugehörige Labels und soll darin selbstständig Strukturen oder Muster entdecken [vgl. 15, S. 3] [vgl. 16, S. 694]. Aufgaben sind beispielsweise das Gruppieren ähnlicher Datenpunkte (*Clustering*), das Reduzieren der Dimensionalität sowie das Schätzen der zugrunde liegenden Datenverteilung $p_{\text{data}}(\mathbf{x})$ [vgl. 15, S. 3] [vgl. 17, S. 146].

Bei der Dichteschätzung wird ein Modell $p_{\theta}(\mathbf{x})$ mit Parametern θ so an die Trainingsdaten angepasst, dass es die echte Datenverteilung möglichst gut nachbildet. Ein verwendetes Lernkriterium hierfür ist die *Maximum-Likelihood-Schätzung*. Es werden die Parameter gewählt, unter denen die Trainingsdaten die höchste Wahrscheinlichkeit besitzen. In der Praxis wird dafür die Log-Likelihood maximiert, da dies numerisch stabiler ist [vgl. 17, S. 131 f.]:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^N \log p_{\theta}(\mathbf{x}_i) \quad (3.2)$$

3.3.3 Bestärkendes Lernen

Beim *reinforced learning* (de: *bestärkendes Lernen*) lernt ein Agent durch direkte Interaktion mit einer Umgebung, welche Handlungen langfristig die größte Belohnung einbringen [vgl. 18, S. 2 f.] [vgl. 16, S. 830]. Zu jedem Zeitschritt t beobachtet der Agent einen Zustand S_t , führt eine Aktion A_t aus und erhält daraufhin eine Belohnung R_{t+1} sowie einen neuen Zustand S_{t+1} [vgl. 18, S. 54].

Diese Wechselwirkung zwischen Agent und Umgebung wird als *Markov Decision Process* (MDP) modelliert. Ein MDP besteht aus einer Menge von Zuständen S , einer Menge von Aktionen A , den Übergangswahrscheinlichkeiten $p(s' | s, a)$, einer Belohnungsfunktion $r(s, a)$ und einem Diskontierungsfaktor $\gamma \in [0, 1]$, der angibt, wie stark zukünftige Belohnungen gegenüber sofortigen gewichtet werden [vgl. 18, S. 67 ff.] [vgl. 16, S. 646 f.].

Die Verhaltensstrategie des Agenten wird durch eine *Policy* $\pi(a | s)$ beschrieben, also die Wahrscheinlichkeit, im Zustand s die Aktion a zu wählen [vgl. 18, S. 54]. Wie gut eine Policy ist, lässt sich über die *Wertfunktion* $V^{\pi}(s)$ messen. Diese gibt die erwartete Summe aller zukünftigen Belohnungen an, wenn der Agent im Zustand s startet und sich an die Policy π hält [vgl. 18, S. 70]:

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \quad (3.3)$$

Die Wertfunktion lässt sich rekursiv über die sogenannte *Bellman-Gleichung* ausdrücken, welche die Grundlage für reinforcement learning-Algorithmen bildet [vgl. 18, S. 71 f.]:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) \left[r(s, a) + \gamma V^\pi(s') \right] \quad (3.4)$$

Ziel des reinforcement learning ist es, eine optimale Policy π^* zu finden, die in jedem Zustand die Wertfunktion maximiert [vgl. 18, S. 75 f.]. Im Gegensatz zum supervised learning erhält der Agent dabei keine direkte Rückmeldung über die korrekte Aktion, sondern muss diese selbstständig herausfinden [vgl. 18, S. 1 f.].

3.4 Artificial Neural Networks

Ein *Artificial Neural Networks* (ANN) (de: *Künstliches Neuronales Netzwerk*) ist ein rechnergestütztes Modell, das die Informationsverarbeitung biologischer Nervensysteme vereinfacht nachbildet [vgl. 19, S. 52] [vgl. 14, S. 82]. Es besteht aus einer Vielzahl miteinander verbundener *künstlicher Neuronen*, die in Schichten organisiert sind: einer Eingabeschicht, einer oder mehreren *verborgenen Schichten* (eng: *hidden layers*) und einer Ausgabeschicht [vgl. 17, S. 168 f.]. Durch das iterative Anpassen der internen Parameter also der *Gewichte* (eng: *weights*) zwischen den Neuronen ist ein ANN in der Lage, komplexe Muster in Daten zu erkennen und Vorhersagen zu treffen [vgl. 14, S. 99]. Damit lassen sich Aufgaben bearbeiten, die zuvor menschliche Intelligenz erforderten, beispielsweise die Bilderkennung oder die Verarbeitung natürlicher Sprache (Natural Language Processing (NLP)) [vgl. 19, S. 52] [vgl. 17, S. 443]. Ein weiterer Vorteil neuronaler Netze ist ihre Robustheit gegenüber verrauschten oder unvollständigen Daten [vgl. 14, S. 83].

3.4.1 Netzwerkarchitekturen

Je nach Verbindungsstruktur der Neuronen lassen sich zwei grundlegende Architekturen unterscheiden [vgl. 17, S. 168]:

- In einem *Feedforward Neural Network* (FNN) werden Informationen ausschließlich in eine Richtung propagiert. Nämlich von der Eingabe- über die verborgenen Schichten zur Ausgabeschicht. Es existieren keine Rückkopplungen zwischen den Schichten [vgl. 17, S. 168].
- In einem *Recurrent Neural Network* (RNN) hingegen existieren Verbindungen, die zu vorherigen Schichten oder zum Neuron selbst zurückführen. Dadurch wird ein interner Zustand realisiert, der die Verarbeitung sequenzieller Daten wie Texte oder Zeitreihen ermöglicht [vgl. 17, S. 374].

Die einfachste Form eines FNN ist das *Perceptron*, das im folgenden Unterabschnitt 3.4.2 vorgestellt wird.

3.4.2 Perceptron

Das Perceptron wurde von Rosenblatt im Jahr 1957 eingeführt und gilt als die einfachste Form eines ANN. Es handelt sich um einen Mechanismus zur Mustererkennung, dessen Ausgaben nicht auf einem rein deterministischen Prozess, sondern auf Wahrscheinlichkeiten beruhen. Ein Modell, das speziell visuelle Muster verarbeitet, wird als *Photoperceptron* bezeichnet [vgl. 20, S. 2].

Mathematisch ist das Perceptron ein lineares Klassifikationsmodell und gehört zu den diskriminativen Modellen [vgl. 21, S. 40]. Es nimmt einen Eingabevektor $x \in \mathbb{R}^n$ entgegen, gewichtet ihn mit einem Gewichtsvektor $w \in \mathbb{R}^n$ und addiert einen Bias-Term $b \in \mathbb{R}$. Auf die resultierende Summe wird eine Aktivierungsfunktion angewendet, sodass sich die Klassifikationsfunktion ergibt zu [vgl. 21, S. 39]:

$$f(x) = \text{sign}(w \cdot x + b) \quad (3.5)$$

Die Aktivierungsfunktion sign ist dabei als Vorzeichenfunktion definiert [vgl. 21, S. 40]:

$$\text{sign}(x) = \begin{cases} +1, & x \geq 0 \\ -1, & x < 0 \end{cases} \quad (3.6)$$

Geometrisch entspricht die Gleichung $w \cdot x + b = 0$ einer Hyperebene im Raum $\mathbb{R}^n$, die diesen in zwei Klassen unterteilt [vgl. 14, S. 86]. Abb. 3.1 fasst diesen Berechnungsablauf schematisch zusammen.

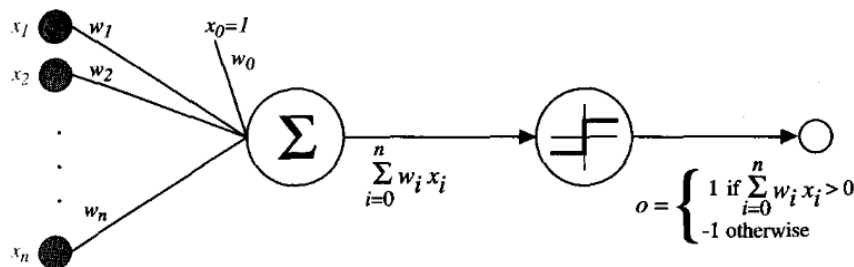


Abb. 3.1: Schematische Darstellung eines Perceptrons. Die Eingaben $x_1, \dots, x_n$ werden mit den Gewichten $w_1, \dots, w_n$ gewichtet aufsummiert. Der Bias-Term ist hierbei als zusätzliche konstante Eingabe $x_0 = 1$ mit zugehörigem Gewicht w_0 realisiert. Auf die gewichtete Summe wird eine Vorzeichenfunktion angewendet, die die Ausgabe $o \in \{+1, -1\}$ liefert [14, S. 87].

Die Gewichte werden iterativ über die sogenannte Perceptron-Lernregel angepasst. Wird eine Eingabe falsch klassifiziert, so erfolgt eine zum Fehler proportionale Korrektur der Gewichte [vgl. 14, S. 88]:

$$w_i^{(t+1)} = w_i^{(t)} + \eta \cdot (y_{\text{soll}} - y_{\text{ist}}) \cdot x_i \quad (3.7)$$

Dabei bezeichnet η die Lernrate, die die Schrittweite der Gewichts Anpassung steuert.

Ein einzelnes Perceptron kann als Boolesche Funktion verwendet werden und so die logischen Operationen **AND**, **OR**, **NAND** und **NOR** darstellen. AND und OR lassen sich

dabei als *m-of-n*-Funktionen auffassen, bei denen mindestens m von n Eingaben wahr sein müssen. Die **XOR**-Funktion lässt sich durch ein einzelnes Perceptron hingegen nicht repräsentieren, da ihre Ausgaben nicht linear separierbar sind und somit keine einzelne Hyperebene die beiden Klassen korrekt trennen kann [vgl. 14, S. 87]. Minsky und Papert zeigten formal, dass diese Einschränkung eine ganze Klasse einfacher logischer Funktionen betrifft [22].

Diese Limitation wird durch das *Multi-Layer Perceptron* (MLP) überwunden, bei dem mehrere Neuronen in verborgenen Schichten angeordnet sind. Bereits ein zweischichtiges Netz aus Perceptronen ist ausreichend, um eine beliebige Boolesche Funktion darzustellen [vgl. 14, S. 105]. In Kombination mit dem Backpropagation-Algorithmus, der den Fehler schichtweise rückwärts durch das Netz propagiert und so die Gewichte aller Schichten anpasst, können auch nichtlineare Zusammenhänge modelliert werden [vgl. 14, S. 97 ff.]. Diese mehrschichtigen Architekturen bilden die Grundlage für die in Kapitel 5 vorgestellten tiefen neuronalen Netze.

3.5 Unterscheidung zwischen Diskriminative und Generative Modelle

Modelle des maschinellen Lernens werden häufig in zwei Kategorien unterteilt: diskriminative und generative Modelle. Beide verfolgen unterschiedliche Ziele und unterscheiden sich darin, was sie aus den gegebenen Daten lernen.

3.5.1 Diskriminative Modelle

Diskriminative Modelle versuchen Eingabedaten einer bestimmten Klasse zuzuordnen. Sie modellieren dazu direkt die Wahrscheinlichkeit $p(y|x)$ des Labels y gegeben der Eingabe x oder erlernen eine direkte Abbildung von der Eingabe x auf das Label y [vgl. 23, S. 1]. Auf diese Weise lernen sie eine Entscheidungsgrenze zwischen den Klassen, ohne die gemeinsame Verteilung $p(x, y)$ zu modellieren [vgl. 15, S. 43].

Beispiele für diskriminative Modelle sind die logistische Regression [vgl. 15, S. 205] sowie Feed-forward Network Functions [vgl. 15, S. 227]. Sie sind dadurch allerdings nicht in der Lage neue Daten zu generieren, da ihnen die Information über die gemeinsame Verteilung $p(x, y)$ fehlt. Ein diskriminatives Modell könnte beispielsweise lernen, ob ein Röntgenbild eine Pathologie zeigt oder nicht, ohne zu verstehen, wie ein Röntgenbild grundsätzlich aufgebaut ist.

3.5.2 Generative Modelle

Generative Modelle hingegen sind in der Lage neue synthetische Daten zu generieren. Sie lernen, wie die Daten strukturiert sind und können durch das Erlernen dieser Struktur neue Daten generieren, die dieser Struktur folgen [vgl. 24, S. 112]. Generative Modelle

lernen dazu die gemeinsame Wahrscheinlichkeitsverteilung $p(x, y)$ der Eingabe x und der zugehörigen Labels y [vgl. 23, S. 1]

Diese Modelle werden auch als *Deep Generative Model* (DGM) bezeichnet. Unter ein DGM wird ein neuronales Netzwerk verstanden, dass die Wahrscheinlichkeitsverteilung von hoch dimensionalen Daten, wie Bilder, Text oder Audio, lernt [vgl. 25, S. 16]. Sie besitzen eine hohe Anzahl von versteckten Schichten aus Neuronen, um das Erlernen der hochdimensionalen Datenverteilungen zu ermöglichen. Allerdings sind sie sehr rechenaufwendig und die Performanz ist stark abhängig von *Hyperparametern*. Hyperparameter beziehen sich auf Einstellungen, die vor dem Training festgelegt werden und den Lernprozess steuern. Dazu gehören die Netzwerkarchitektur, die Wahl der Zielfunktion, Regularisierungstechniken sowie Trainingsalgorithmen [vgl. 26, S. 2]. Somit erlernen generative Modelle wie beispielsweise ein Röntgenbild aufgebaut ist und auf dieser Basis neue, synthetische Röntgenbilder erzeugen.

3.6 Arten der Generierung

Generative Modelle lassen sich dahingehend unterscheiden, ob der Generierungsprozess durch zusätzliche Informationen gesteuert wird. In diesem Zusammenhang wird zwischen *unconditional* und *conditional* Generierung differenziert.

3.6.1 Unconditional

Bei der *unconditional* (de: *bedingungslos*) Generierung lernt ein generatives Modell die Datenverteilung $p(\mathbf{x})$ ohne Annotationen. Dabei werden Samples allein aus der erlernten Datenverteilung gezogen. Allerdings schneidet die unbedingte Generierung schlechter ab, da im konditionalen Fall durch menschliche Annotationen zusätzliche semantische Information bereitgestellt wird [27, S. 3]. Methodisch bildet die unbedingte Generierung zugleich die Grundlage konditionaler Modelle, da diese auf denselben Trainings- und Sampling-Mechanismen aufbauen [28, S. 16].

3.6.2 Conditional

Bei der *conditional* (de: *konditioniert*) Generierung wird der Generierungsprozess durch eine zusätzliche Information $\mathbf{y}$ gesteuert, sodass das Modell die bedingte Verteilung $p(\mathbf{x} | \mathbf{y})$ lernt. Als Konditionierung dienen je nach Anwendungsfall beispielsweise textuelle Beschreibungen oder Bilder [vgl. 28, S. 5 ff.].

Jeodch ist das einfache Konditionieren auf $\mathbf{y}$ häufig nicht ausreichend, um eine deutliche Ausrichtung der erzeugten Samples an der Bedingung zu erzielen. Deshalb wird der Einfluss der Konditionierung beim Sampling künstlich verstärkt. Dieses Vorgehen wird als *Guidance* bezeichnet [vgl. 28, S. 17]. Dabei gibt es zwei Verfahren: *Classifier Guidance* [29] und *Classifier-Free Guidance* [30]. Classifier Guidance verwendet hierfür den Gradienten $\nabla_{\mathbf{x}_t} \log p_{\phi}(\mathbf{y} | \mathbf{x}_t)$ eines separat auf verrauschten Daten trainierten Classifiers, was allerdings das aufwendige Training eines zusätzlichen Modells erfordert. Der

so gewonnene Gradient wird beim Sampling zur geschätzten Score-Funktion addiert und lenkt den Prozess in Richtung der Bedingung $\mathbf{y}$ [vgl. 29, S. 6 f.].

Classifier-Free Guidance kommt ohne einen weiteren Classifier aus. Ein einzelnes Netzwerk wird gemeinsam für die bedingte und die unbedingte Schätzung trainiert, indem die Bedingung $\mathbf{y}$ während des Trainings mit einer festen Wahrscheinlichkeit durch ein Null-Token $\emptyset$ ersetzt wird [vgl. 30, S. 4]. Beim Sampling werden die bedingte und die unbedingte Rauschvorhersage linear kombiniert [30, S. 5]:

$$\tilde{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}) = (1 + w) \epsilon_{\theta}(\mathbf{x}_t, \mathbf{y}) - w \epsilon_{\theta}(\mathbf{x}_t, \emptyset), \quad (3.8)$$

wobei $w \geq 0$ die Guidance-Stärke bezeichnet. Über die Bayes-Beziehung $\nabla_{\mathbf{x}_t} \log p(\mathbf{y} | \mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{y}) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$ lässt sich diese Linearkombination als implizite Approximation des Classifier-Gradienten interpretieren, sodass auf ein zusätzliches Modell verzichtet werden kann [vgl. 30, S. 2] [vgl. 28, S. 18]. Aufgrund dieser Einfachheit wird Classifier-Free Guidance heute häufig für das Training konditionaler Diffusionsmodelle von Grund auf eingesetzt. Jedoch ist das Verfahren wegen des damit verbundenen Rechenaufwands nicht für die Adaption oder das Fine-Tuning bereits trainierter Modelle geeignet [vgl. 28, S. 17].

Die Guidance-Stärke w steuert dabei das Ergebnis. Bei $w = 0$ entspricht das Sampling der rein bedingten Vorhersage, während größere Werte die Treue zur Bedingung erhöhen, jedoch mit einer Reduktion der Sample-Diversität einhergehen [vgl. 28, S. 17 ff.].

3.7 Einordnung der Diffusionsmodelle

Mit den bisher eingeführten Konzepten lassen sich Diffusionsmodelle nun methodisch einordnen. Sie gehören zur schwachen KI, da sie auf einen klar abgegrenzten Anwendungsbereich, wie der Generierung medizinischer Bilddaten, zugeschnitten sind und sich nicht direkt auf andere Domänen übertragen lassen. Gleichzeitig zählen sie zur sub-symbolischen KI. Diffusionsmodelle repräsentieren ihr Wissen verteilt in den Gewichten tiefer neuronaler Netze und lernen die gegebene Datenverteilung induktiv anhand von Beispielen, ohne dass explizite Regeln zur Bildgenerierung vorgegeben werden.

Innerhalb des maschinellen Lernens gehören Diffusionsmodelle zu den generativen Modellen, da sie die gegebene Wahrscheinlichkeitsverteilung der Trainingsdaten approximieren und daraus neue, synthetische Samples erzeugen können. Wegen ihrer hohen Modelltiefe und der Verarbeitung hochdimensionaler Bilddaten lassen sie sich genauer den DGM zuordnen. Mehr dazu in Kapitel 5. Weiterhin können Diffusionsmodelle unconditional arbeiten, wie zum Beispiel der einfachen Generierung von Samples oder conditional, indem die Generierung gezielt durch zusätzliche Informationen wie Klassenlabels oder vorhandene Bilddaten gesteuert wird.

4 Stand der Forschung

In diesem Kapitel wird die Forschungslage zur Generierung dreidimensionaler medizinischer Bilddaten beschrieben. Dabei werden zentrale Arbeiten zu GAN-, VAE- und Diffusion-basierten Ansätzen vorgestellt.

4.1 GAN-basierte Ansätze

Frühe Arbeiten zur generativen Modellierung medizinischer 3D-Volumina entstanden zunächst im Bereich der cross-modalen Bildtranslation. Also Netzwerke, die ein Volumen einer Modalität (z. B. MRT) in das entsprechende Volumen einer anderen Modalität (z. B. CT) überführen. Zhang, Yang und Zheng stellten 2018 mit ihrer 3D-CycleGAN-basierten Architektur eine der ersten dieser Arbeiten vor, die auf 4.496 kardiovaskulären CT- und MRT-Volumina trainiert wurde [vgl. 31, S. 1]. Daraufhin schlugen Yu u. a. mit ihrem 3D-conditional Generative Adversary Network (cGAN) ein konditioniertes Modell zur cross-modalen MRT-Synthese vor, das in der Folgezeit häufig als frühe 3D-GAN-Baseline herangezogen wird [vgl. 32, S. 1]. Diese Arbeiten erzeugten Volumina nicht aus zufälligem Rauschen, sondern transformierten existierende Eingaben. Sie verarbeiteten volumetrische Daten direkt und unterscheiden sich damit von der zuvor verbreiteten Strategie, einzelne 2D-Schichten zu generieren und anschließend zu einem Volumen zusammenzusetzen, was zu Diskontinuitäten zwischen den Schichten führen konnte [vgl. 33, S. 2f.].

Den ersten Ansatz zur Generierung vollständiger 3D-Hirn-MRT-Volumina aus zufälligem Rauschen stellten Kwon, Han und Kim mit dem 3D- α -WGAN-GP vor. Es ist ein erweitertes GAN-Modell, das um einen Encoder und eine stabilere Verlustfunktion erweitert wurde, um typische Probleme beim Training von GAN wie Mode Collapse und Trainingsinstabilität zu reduzieren. Die Experimente wurden auf eine Auflösung von 64^3 Voxeln beschränkt [vgl. 34, S. 4]. Sun u. a. adressierten dieses Speicherproblem mit einem Hierarchical Amortized Generative Adversary Network (HA-GAN). Während des Trainings werden parallel ein niedrig aufgelöstes Gesamtvolumen und ein zufällig gewähltes hoch aufgelöstes Teilvolumen generiert, sodass der Speicherbedarf auf Teilvolumina verteilt wird. Zur Inferenzzeit wird das vollständige 256^3 -Volumen erzeugt [vgl. 35, S. 2 f.]. Trotz dieser Fortschritte bleiben GAN-basierte Ansätze anfällig für Trainingsinstabilitäten und Mode Collapse [vgl. 36, S. 2].

4.2 VAE-basierte Ansätze

Parallel zu den GAN-basierten Ansätzen wurde auch der Einsatz von VAE für die 3D-Bildsynthese untersucht. Volokitin et al. schlugen einen schichtweisen Ansatz vor, bei

dem ein 2D-Slice-VAE mit einem Gauß-Modell kombiniert wird, um die Abhängigkeiten zwischen benachbarten Schichten zu modellieren und so kohärente 3D-Hirn-MRT-Volumina zu erzeugen [vgl. 37, S. 3]. VAE-basierte Modelle leiden jedoch typischerweise unter unscharfen Samples [vgl. 38, S. 2].

4.3 Diffusionsbasierte Ansätze

Allerdings stellten sich Diffusionsmodelle als erfolgreicher heraus. Pinaya u. a. präsentierten eine der ersten Anwendungen eines Latent Diffusion Model (LDM) auf hochaufgelöste 3D-Hirn-MRT und trainierten ihr Modell auf 31 740 T1-gewichteten Aufnahmen der UK-Biobank-Datenbank. Aus dem trainierten Modell wurde ein synthetischer Datensatz von 100 000 Volumina abgeleitet und veröffentlicht [vgl. 39, S. 1]. Im Vergleich zu GAN-basierten Baselines erzielte das LDM mit einem FID von 0,0076 deutlich realistischere Bilder als VAE-GAN (0,1576) und Last Squares Generative Adversarial Networks (LSGAN) (0,0231). Zudem lagen die Diversitätsmetriken nahe an der realen Datenverteilung, während die GAN-Modelle unter Mode Collapse litten [vgl. 39, S. 5]. Khader u. a. erweiterten diesen Ansatz und legten die erste systematische Evaluation eines LDM für volumetrische MRT- und CT-Daten vor. Sie trainierten ihr Modell auf vier verschiedenen Datensätzen (Hirn-MRT, Brust-MRT, Knie-MRT und Thorax-CT) und nutzten ein Vector Quantized Generative Adversarial Network (VQ-GAN) als Encoder [vgl. 36, S. 2]. Die Ergebnisse zeigten, dass LDM besser sind gegenüber GAN-Ansätzen auf mehreren Ebenen. Im direkten Vergleich mit einem α -WGAN-GP-Modell nach Kwon, Han und Kim erreichte das Diffusionsmodell auf den Hirn-MRT einen MS-SSIM-Wert von 0,8557, dass nahe an dem der realen Daten, mit einem Wert von 0,8095, ist. Dabei zeigte das α -WGAN-GP-Modell mit 0,9996 einen Mode Collapse [vgl. 36, S. 11]. Zwei erfahrene Radiologen beurteilten die generierten Volumina anhand der Kategorien realistisches Erscheinungsbild, anatomische Korrektheit und Schichtkonsistenz und stuften sie mehrheitlich als realistisch ein [vgl. 36, S. 8]. Schließlich zeigen die Autoren auch die praktische Nutzbarkeit der synthetischen Daten. Im Self-Supervised Pre-Training eines Segmentierungsmodells stieg der DSC in einem Szenario von 0,91 auf 0,95 [vgl. 36, S. 11]. Beide Arbeiten zeigen, dass mit Diffusionsmodellen die Erzeugung realistischer 3D-medizinischer Bilddaten möglich ist und somit als Grundlage dient, auf der sich synthetische Datensätze für Forschung, Datenschutz und das Training weiterer Modelle aufbauen lassen.

Weiterhin wurden auch konditionierte Modelle entwickelt, um die Bilderzeugung gezielter zu steuern. Dorjsembe u. a. stellten mit *Med-DDPM* das erste Diffusionsmodell zur semantischen 3D-Hirn-MRT-Synthese vor. Es wurde eine eine Konditionierungsmaske über eine kanalweise Konkatenation in das Modell eingebunden, sodass die Position pathologischer Strukturen gezielt vorgegeben werden kann [vgl. 40, S. 3]. In der Tumorsegmentierung erreichten die rein synthetischen Bilder einen DSC von 0,6207, der nahe am Wert der realen Daten mit 0,6531 lag und den Baseline-Modell übertraf. durch Kombination synthetischer mit realen Bildern stieg der DSC auf 0,6675, was die Möglichkeit für die Datenaugmentation belegt [vgl. 40, S. 9 f.].

5 Deep Generative Model

Deep Generative Model (DGM) approximieren die Verteilung eines Datensatzes, um daraus neue Samples zu erzeugen. Die folgenden Abschnitte stellen mit GAN, VAE und Diffusionsmodellen drei zentrale Ansätze vor und behandeln abschließend mit dem CNN und der U-Net-Architektur die zugrunde liegenden Architekturkonzepte.

5.1 Generative Adversarial Networks

Generative Adversary Network (GAN) wurden von Goodfellow u. a. eingeführt und bestehen aus zwei neuronalen Netzwerken, die in einem adversarialen Prozess gegeneinander trainiert werden. Diese beiden Netzwerke werden als *Generator* G und *Diskriminator* D bezeichnet. Der Generator bildet einen latenten Rauschvektor $z \sim p_z(z)$ auf ein synthetisches Sample $G(z)$ ab, während der Diskriminator lernt, echte Datenpunkte $x \sim p_{data}(x)$ von gefälschten zu unterscheiden. Dieses Wettbewerbsprinzip lässt sich formal als *Minimax-Spiel* mit Wertfunktion $V(D, G)$ formulieren [vgl. 41, S. 2 f.]. Die mathematische Formulierung lautet [41, S. 3]:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (5.1)$$

Der Diskriminator maximiert diesen Ausdruck, indem er $D(x) \rightarrow 1$ für echte und $D(G(z)) \rightarrow 0$ für generierte Proben anstrebt. Der Generator minimiert ihn dagegen, indem er versucht, D zu täuschen. Goodfellow u. a. zeigen, dass dieses Spiel ein globales Optimum bei $p_g = p_{data}$ besitzt, an dem $D(x) = \frac{1}{2}$ gilt und der Diskriminator echte und synthetische Daten nicht mehr unterscheiden kann [vgl. 41, S. 4 f.].

5.2 Variational Autoencoders

Variational Autoencoder Network (VAE) von Kingma und Welling verfolgen einen probabilistischen Ansatz. Sie modellieren die Daten als gemeinsame Verteilung $p_\theta(x, z) = p_\theta(z) p_\theta(x | z)$ über beobachtete Variablen x und latente Variablen z mit der A-priori-Verteilung $p_\theta(z)$ (typischerweise $N(0, I)$). Da sich die wahre A-posteriori-Verteilung $p_\theta(z | x)$ in der Regel nicht in geschlossener Form berechnen lässt, wird sie durch eine parametrisierte Verteilung $q_\phi(z | x)$, das sogenannte *Erkennungsmodell* (auch *Encoder* genannt), angenähert. Der *Decoder* $p_\theta(x | z)$ rekonstruiert dann x aus z [vgl. 42, S. 2 f.].

Das Training maximiert nicht direkt die Log-Likelihood $\log p_\theta(x)$, sondern die Evidence Lower Bound (ELBO) $L(\theta, \phi; x)$, eine untere Schranke für die Log-Likelihood [42, S. 3]:

$$L(\theta, \phi; x^{(i)}) = -D_{KL}(q_\phi(z | x^{(i)}) \| p_\theta(z)) + \mathbb{E}_{q_\phi(z | x^{(i)})} [\log p_\theta(x^{(i)} | z)] \quad (5.2)$$

Der erste Term wirkt als Regularisierer, der die genäherte A-posteriori-Verteilung $q_\phi(z | x)$ in Richtung der A-priori-Verteilung $p_\theta(z)$ zieht. Der zweite Term beschreibt die erwartete Rekonstruktionsqualität und sorgt dafür, dass aus dem latenten Code z die ursprüngliche Eingabe rekonstruiert werden kann [vgl. 42, S. 3 f.].

Da die Erwartungswertbildung in Gleichung (5.2) bezüglich ϕ nicht direkt differenzierbar ist, verwenden Kingma und Welling den *Reparametrisierungstrick*. Ein Sample $z \sim q_\phi(z | x) = N(\mu_\phi(x), \sigma_\phi^2(x))$ wird als deterministische Funktion eines Hilfsrauschens $\epsilon \sim N(0, I)$ ausgedrückt [vgl. 42, S. 4 f.],

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad (5.3)$$

wodurch der Gradient der ELBO per Standard-Backpropagation durch das Netzwerk propagiert werden kann [vgl. 42, S. 4].

5.3 Diffusion Model

Ein *Diffusion Model*, im Deutschen meist *Diffusionsmodell*, ist ein generatives Modell, das zur Klasse der latenten Variablenmodelle gehört [vgl. 43, S. 153, 24, S. 114]. Das grundlegende Prinzip besteht darin, einen Diffusionsprozess umzukehren, bei dem schrittweise Rauschen zu den Daten hinzugefügt wird. Durch die Umkehrung dieses Prozesses kann das Modell ausgehend von reinem Rauschen schrittweise neue Datenpunkte generieren. Dafür wird eine parametrisierte Markov-Kette mittels Variationsinferenz trainiert, deren Übergänge lernen, den Diffusionsprozess umzukehren [vgl. 43, S. 153, 44, S. 2].

5.3.1 Markov-Ketten

Eine Markov-Kette ist ein stochastischer Prozess, bei dem der zukünftige Zustand ausschließlich vom aktuellen Zustand abhängt und nicht von der Vergangenheit. Diese Eigenschaft wird als *Gedächtnislosigkeit* bezeichnet. Für eine Sequenz von Zufallsvariablen $x_0, x_1, \dots, x_T$ lässt sich dies ausdrücken als [vgl. 15, S. 608]:

$$p(x_t | x_0, \dots, x_{t-1}) = p(x_t | x_{t-1}), \quad t = 1, \dots, T \quad (5.4)$$

Hierbei bezeichnet t den Zeitschritt und x_t den Zustand zum Zeitpunkt t . Eine solche Markov-Kette, bei der der aktuelle Zustand nur vom unmittelbar vorhergehenden abhängt, wird als Markov-Kette *erster Ordnung* bezeichnet [vgl. 15, S. 609]. In Abb. 5.1 wird diese graphisch dargestellt. Aufgrund der Markov-Eigenschaft kann die gemeinsame Wahrscheinlichkeit der gesamten Sequenz $x_{1:T}$ gegeben x_0 als Produkt der einzelnen Übergangswahrscheinlichkeiten faktorisiert werden [vgl. 15, S. 607]:

$$p(x_{1:T} | x_0) = \prod_{t=1}^T p(x_t | x_{t-1}) \quad (5.5)$$

Diese Faktorisierung ist für Diffusionsmodelle von zentraler Bedeutung [vgl. 43, S. 153 f.] [vgl. 15, S. 608 f.]. Die Gedächtnislosigkeit der Markov-Kette erweist sich als vorteilhaft

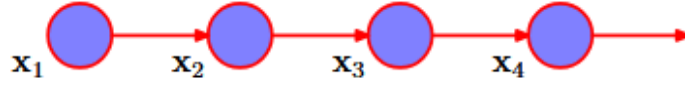


Abb. 5.1: Markov-Kette erster Ordnung [15, S. 609]

für Diffusionsprozesse, da jeder Schritt sowohl beim Hinzufügen als auch beim Entfernen von Rauschen ausschließlich vom aktuellen Zustand abhängt.

5.3.2 Noise-Schedule

Der *Noise Schedule* definiert eine Sequenz von vorgegebene Werten $\beta_1, \beta_2, \dots, \beta_T$ mit $\beta_t \in (0,1)$, die steuern wie viel Rauschen in jedem Schritt t hinzugefügt wird. Aus diesen werden zwei Werte definiert [25, S. 43 f.] [43, S. 155 f.]:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{k=1}^t \alpha_k \quad (5.6)$$

Dabei gibt α_t an, welcher Anteil des Signals aus dem vorherigen Schritt erhalten bleibt. Hingegen tut $\bar{\alpha}_t$ das kumulative Produkt über alle bisherigen Schritte darstellen. Mit steigendem t nimmt $\bar{\alpha}_t$ monoton ab, sodass das ursprüngliche Signal zunehmend durch Rauschen überschrieben wird [vgl. 25, S. 44]. Heißt, wenn für $t \rightarrow T$ gilt $\bar{\alpha}_T \approx 0$, was bedeutet, dass das Originalbild nahezu vollständig zerstört ist.

Die Wahl des Noise Schedulers beeinflusst die Qualität der generierten Daten. Der *linear Schedule*, der von Ho, Jain und Abbeel verwendet wird, definiert β_t als lineare Interpolation zwischen einem Start- und Endwert [vgl. 44, S. 5]:

$$\beta_t = \beta_1 + \frac{t-1}{T-1}(\beta_T - \beta_1) \quad (5.7)$$

Allerdings wurde beobachtet, dass der lineare Schedule in den späten Schritten zu aggressiv rauscht. Nichol und Dhariwal verwenden daher einen *Cosine Schedule*, der $\bar{\alpha}_t$ direkt über eine Kosinusfunktion definiert [vgl. 45, S. 4]:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad \text{mit} \quad f(t) = \cos\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)^2 \quad (5.8)$$

wobei s ein kleiner Offset-Parameter ist, der verhindert, dass β_t in der Nähe von $t = 0$ zu klein wird. Der Cosine Schedule führt zu einem gleichmäßigeren Rauschverlauf und verbessert die Bildqualität insbesondere bei niedrigen Auflösungen.

TODO: Vllt. Bild hinzufügen aus Nichol?

5.3.3 Vorwärtsprozess

Die Abb. 5.2 stellt den Vorwärts- und Rückwärtsprozess an einem Bild dar. Der Vorwärts-

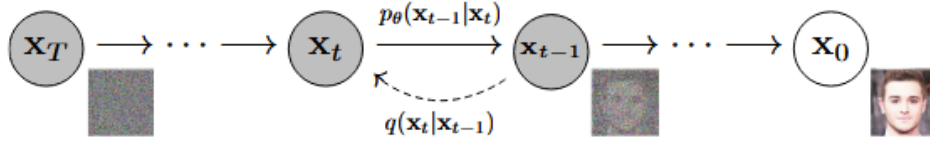


Abb. 5.2: Darstellung der Vorwärts- und Rückwärtsprozesse

prozess (engl. *Forward Process*) ist fest vorgegeben, wird nicht trainiert und dient als Encoder. Er fügt einem Signal x_0 schrittweise Gaußsches Rauschen hinzu bis am Ende reines Rauschen übrig ist. Die Übergangsverteilung für einen einzelnen Schritt ist definiert als [vgl. 43, S. 155 f.]:

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right) \quad (5.9)$$

Hierbei wird das Signal aus dem vorherigen Schritt mit dem Faktor $\sqrt{1 - \beta_t}$ skaliert und Gaußsches Rauschen mit Varianz β_t hinzugefügt. Wobei eben $0 < \beta_1 < \beta_T < 1$ gilt. Da jeder Schritt nur vom vorherigen Zustand abhängt, kann die gemeinsame Verteilung des gesamten Vorwärtsprozesses gemäß der Markov-Eigenschaft faktorisiert werden mit [vgl. 43, S. 156] [vgl. 44, S. 2]:

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}) \quad (5.10)$$

Dies bedeutet, dass ein Bild x_0 nach T Schritten vollständig zerstört wird und das Ergebnis x_T reinem Gaußschem Rauschen $\mathcal{N}(0, I)$ entspricht. Ein Vorteil des Vorwärtsprozesses ist, dass x_t nicht iterativ berechnet werden muss. Mithilfe der Definitionen $\alpha_t = 1 - \beta_t$ und $\bar{\alpha}_t = \prod_{k=1}^t \alpha_k$ lässt sich x_t direkt aus dem Originalbild x_0 berechnen [vgl. 43, S. 156] [vgl. 44, S. 2] [vgl. 25, S. 44 f.]:

$$q(x_t | x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I\right) \quad (5.11)$$

Daraus ergibt sich die folgende Formel [43, S. 156] [44, S. 2] [25, S. 44 f.]:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (5.12)$$

Dadurch kann direkt zu einem beliebigen Rauschzustand x_t gesprungen werden ohne alle Zwischenschritte $x_0, x_1, \dots, x_{t-1}$ durchlaufen zu müssen. Dies ermöglicht dann ein effizientes, da zufällige Zeitschritte t gesampelt werden können [vgl. 44, S. 2 f.].

5.3.4 Rückwärtsprozess

Der Rückwärtsprozess (engl. *Reverse Process*) ist das Gegenstück zum Vorwärtsprozess und dient als lernbarer Decoder. Er startet bei reinem Rauschen $x_T \sim \mathcal{N}(0, I)$ und entfernt in jedem Schritt einen Teil des Rauschens, bis ein sinnvolles Datensample x_0 rekonstruiert ist. Auch dieser Prozess wird als Markov-Kette dargestellt [vgl. 44, S. 2]:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t) \quad (5.13)$$

Da die wahren Übergangswahrscheinlichkeiten $q(x_{t-1} | x_t)$ von der unbekannten Datenverteilung abhängen, wird jeder Rückwärtsschritt durch ein neuronales Netz mit Parametern θ approximiert [vgl. 44, S. 2]:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I) \quad (5.14)$$

Parametrisierung des Mittelwerts

Anstatt den Mittelwert μ_θ direkt zu schätzen, wird das Netzwerk darauf trainiert, das hinzugefügte Rauschen $\epsilon_\theta(x_t, t)$ vorherzusagen. Aus der Gl. (5.12) des Vorwärtsprozesses lässt sich der Mittelwert dann wie folgt berechnen [vgl. 44, S. 4]

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \quad (5.15)$$

Diese Parametrisierung ist vorteilhaft, da die Vorhersage des Rauschens stabiler zu optimieren ist als die direkte Mittelwertschätzung. Damit ergibt sich der vollständige Sampling-Schritt im Rückwärtsprozess als [vgl. 44, S. 4]:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I) \quad (5.16)$$

Die Varianz σ_t^2 wird üblicherweise als fester Wert gesetzt, wobei $\sigma_t^2 = \beta_t$ oder $\sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$ verwendet wird und für z entweder $z \sim \mathcal{N}(0, I)$ oder $z = 0$ gilt, wenn es für den letzten Schritt $t = 1$ gilt [vgl. 44, S. 4].

5.3.5 Sampling

Das Sampling beschreibt den Prozess der Datengenerierung, bei dem von reinem Rauschen $x_T \sim \mathcal{N}(0, I)$ schrittweise der Rückwärtsprozess durchlaufen wird [vgl. 44, S. 4]. Die Wahl des Sampling-Verfahrens beeinflusst sowohl die Qualität der generierten Daten als auch die benötigte Rechenzeit.

Diffusion Denoising Probabilistic Model

Das DDPM-Sampling implementiert den stochastischen Rückwärtsprozess aus Gl. (5.16) und entspricht damit der ursprünglichen Sampling-Prozedur von Ho, Jain und Abbeel. Ausgehend von reinem gaußschen Rauschen $x_T \sim \mathcal{N}(0, I)$ wird in jedem Schritt $t \rightarrow t - 1$ eine kleine Menge des in x_t enthaltenen Rauschens entfernt, wobei das trainierte Netzwerk $\epsilon_\theta(x_t, t)$ diese Rauschkomponente vorhersagt. Der erste Klammerausdruck aus Gl. (5.16) entspricht dem geschätzten Mittelwert der Rückwärtsverteilung $p_\theta(x_{t-1} | x_t)$, während über das hinzugefügte Rauschen $\sigma_t z$ mit $z \sim \mathcal{N}(0, I)$ die für den Markov-Prozess charakteristische Stochastizität eingebracht wird [vgl. 44, S. 4]. Für die Varianz kann $\sigma_t^2 = \beta_t$ gewählt werden [vgl. 44, S. 3]. Der Prozess durchläuft alle T Zeitschritte,

wobei in der Praxis $T = 1000$ mit einem linearen β -Schedule von $\beta_1 = 10^{-4}$ bis $\beta_T = 0,02$ verwendet wird [vgl. 44, S. 5]. Da bei jedem dieser Schritte eine vollständige Auswertung des neuronalen Netzes erforderlich ist, liefert das Verfahren zwar qualitativ hochwertige Ergebnisse, ist jedoch rechenintensiv und in der Praxis deutlich langsamer als alternative generative Modelle [vgl. 46, S. 3].

Diffusion Denoising Implicit Model

Im Gegensatz zum DDPM-Sampling müssen beim DDIM-Sampling nicht alle T Schritte durchlaufen werden. Dies wird dadurch ermöglicht, dass der Vorwärtsprozess auch als nicht-Markov-Prozess interpretiert werden kann [vgl. 46, S. 3]. Dadurch lassen sich Teilsequenzen der Zeitschritte verwenden, und über den Parameter σ_t kann zusätzlich der Grad der Stochastizität eingestellt werden, bis zu einem vollständig deterministischen Sampling [vgl. 46, S. 5]. Das DDIM-Sampling wird wie folgt beschrieben [vgl. 46, S. 5]:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(x_t, t) + \sigma_t z \quad (5.17)$$

Der erste Term entspricht der Vorhersage des sauberen Bildes x_0 ausgehend vom ver-rauschten Zustand x_t , der zweite Term stellt die deterministische Richtung dar, die auf x_t zeigt, und der letzte Term $\sigma_t z$ mit $z \sim \mathcal{N}(0, I)$ steuert über σ_t den Grad der Stochas-tizität des Verfahrens. Für $\sigma_t = 0$ wird der Prozess deterministisch, das heißt derselbe Startpunkt x_T erzeugt immer dasselbe Ergebnis [vgl. 46, S. 5]. In der Praxis reichen häu-fig 20 bis 100 Schritte statt $T = 1000$, was eine 10- bis 50-fache Beschleunigung bei ver-gleichbarer Bildqualität ermöglicht [vgl. 46, S. 7].

DPM-Solver++

DPM-Solver++ ist ein numerischer Löser für die zum Diffusionsprozess gehörende Dif-ferentialgleichung (Diffusion Ordinary Differential Equation (ODE)) und stellt eine Wei-terentwicklung des ursprünglichen DPM-Solvers dar [vgl. 47, S. 2]. Grundidee dieses Ver-fahrens ist es, den Sampling-Prozess als Lösen dieser ODE zu interpretieren und durch eine Variablentransformation auf das log-SNR $\lambda_t = \log(\alpha_t/\sigma_t)$ den linearen Anteil der Lö-sung analytisch zu berechnen. Lediglich der nichtlineare, durch das Neuronale Netz pa-parametrisierte Anteil muss noch durch eine Taylor-Entwicklung approximiert werden [vgl. 47, S. 4 f.]. Im Gegensatz zum ursprünglichen DPM-Solver, der hierfür das Rauschvor-hersagemodell ϵ_θ verwendet, basiert DPM-Solver++ auf dem Datenvorhersagemodell x_θ , welches direkt das saubere Bild x_0 schätzt. Dies reduziert den Diskretisierungsfehler bei großen Guidance-Skalen und erlaubt zudem den Einsatz von Thresholding-Methoden, die den vorhergesagten Wert auf den gültigen Datenbereich beschränken [vgl. 47, S. 5 ff.].

In der Praxis kommt überwiegend die Multistep-Variante zweiter Ordnung DPM-Solver++(2M) zum Einsatz. Ein Solver zweiter Ordnung berücksichtigt in der Taylor-Entwicklung zusätzlich die erste Ableitung des Vorhersagemodells und erreicht so einen geringeren Diskretisierungsfehler pro Schritt als ein Solver erster Ordnung. Die benötigte

Ableitung wird dabei aus zwischengespeicherten Modellauswertungen vorheriger Schritte approximiert, sodass pro Iteration nur eine einzige Auswertung des Neuronalen Netzes erforderlich ist [vgl. 47, S. 6 f.]. Mit der Schrittweite $h_i := \lambda_{t_i} - \lambda_{t_{i-1}}$ und dem Verhältnis $r_i := h_{i-1}/h_i$ ergibt sich der Update-Schritt zu:

$$\tilde{x}_{t_i} = \frac{\sigma_{t_i}}{\sigma_{t_{i-1}}} \tilde{x}_{t_{i-1}} - \alpha_{t_i} (e^{-h_i} - 1) D_i, \quad D_i = \left(1 + \frac{1}{2r_i}\right) x_\theta(\tilde{x}_{t_{i-1}}, t_{i-1}) - \frac{1}{2r_i} x_\theta(\tilde{x}_{t_{i-2}}, t_{i-2}) \quad (5.18)$$

Der erste Iterationsschritt wird durch eine Aktualisierung erster Ordnung initialisiert, die dem deterministischen DDIM-Sampling entspricht [vgl. 47, S. 9]. Aus der Forschung erzeugt DPM-Solver++(2M) bereits in 15 bis 20 Schritten Bilder vergleichbarer Qualität wie ein DDIM-Sampling mit mehreren hundert Schritten [vgl. 47, S. 11].

Unified Predictor-Corrector

UniPC ist ein trainingsfreies Sampling-Verfahren, das insbesondere bei sehr wenigen Schritten (< 10) eine bessere Sampling-Qualität als vergleichbare Verfahren erreicht [vgl. 48, S. 1 f.]. Die Grundidee besteht darin, jeden Sampling-Schritt in zwei Teilschritte aufzuteilen. Zuerst erzeugt der Prädiktor (UniP) analog zum DPM-Solver++ eine erste Schätzung des nachfolgenden Zustands $\tilde{x}_{t_i}$. Anschließend verbessert der Korrektor (UniC) diese Schätzung zu einem korrigierten Wert $\tilde{x}_{t_i}^c$, indem die Modellauswertung am vorhergesagten Punkt als zusätzlicher Stützpunkt einbezogen wird [vgl. 48, S. 2]. Dadurch erhöht sich die Genauigkeitsordnung des Verfahrens um eins [vgl. 48, S. 4]. Der wesentliche Unterschied zu klassischen Predictor-Corrector-Verfahren aus der Mathematik liegt darin, dass UniC keine zusätzlichen Auswertungen des neuronalen Netzes benötigt. Die im Korrekturschritt berechnete Modellauswertung wird im darauffolgenden Sampling-Schritt wiederverwendet, sodass pro Iteration nur eine einzige Netzauswertung erforderlich ist [vgl. 48, S. 5].

Wie der DPM-Solver++ basiert UniPC auf der Lösung der Diffusion-ODE im log-SNR-Bereich. Mit der Schrittweite $h_i := \lambda_{t_i} - \lambda_{t_{i-1}}$ und den Differenzen D_m aus zusätzlichen Modellauswertungen ergibt sich der Korrekturschritt UniC-p zu:

$$\tilde{x}_{t_i}^c = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}}^c - \sigma_{t_i} (e^{h_i} - 1) \epsilon_\theta(\tilde{x}_{t_{i-1}}, t_{i-1}) - \sigma_{t_i} B(h_i) \sum_{m=1}^p \frac{a_m}{r_m} D_m \quad (5.19)$$

Die ersten beiden Terme entsprechen einer Aktualisierung erster Ordnung, die dem deterministischen DDIM-Sampling entspricht. Der Summenterm bringt die Korrektur höherer Ordnung ein, indem die Differenzen D_m vorangegangener Modellauswertungen mit den Koeffizienten a_m gewichtet kombiniert werden [vgl. 48, S. 3 f.]. Der Prädiktor UniP-p ergibt sich aus Gl. (5.19) durch Weglassen des Terms D_p , da dieser die im Prädiktorschritt noch nicht verfügbare Modellauswertung am aktuellen Zeitschritt voraussetzt. Die Kombination beider Komponenten erreicht eine Genauigkeitsordnung von $p + 1$. Da Prädiktor und Korrektor derselben analytischen Form folgen, lassen sich beliebige Ordnungen einheitlich formulieren, während der DPM-Solver++ nur bis zur dritten Ordnung explizit angegeben werden kann [vgl. 48, S. 5].

5.4 Grundlegende Architekturen

Sowohl der Rückwärtsprozess als auch die vorgestellten Sampling-Verfahren basieren auf einem neuronalen Netzwerk $\epsilon_\theta(x_t, t)$, das in jedem Schritt das im verrauschten Bild x_t enthaltene Rauschen schätzt. Die hierfür eingesetzten Architekturen werden in den folgenden Abschnitten vorgestellt. Den Ausgangspunkt bildet das CNN als Grundlage faltungsbasierter Bildverarbeitung, gefolgt von der darauf aufbauenden U-Net-Architektur, die sich als Standard für Diffusionsmodelle durchgesetzt hat [vgl. 49, S. 2].

5.4.1 Convolutional Neural Network

Convolutional Neural Network (CNN) ist eine Klasse neuronaler Netzwerke, die speziell für die Verarbeitung von Bilddaten entwickelt wurde [vgl. 50, S. 1]. Dies geschieht durch sogenannte Faltungsschichten (engl. *convolutional layers*), die mithilfe von Filtern lokale Muster in der Datenstruktur erkennen. Diese Merkmalsextraktion ermöglicht es dem Modell, komplexe Strukturen mit hoher Genauigkeit zu identifizieren [vgl. 51, S. 1170].

Die Faltung (engl. *convolution*) ist eine mathematische Operation, bei der ein Faltungskern (engl. *kernel*) schrittweise über ein Bild geschoben wird. An jeder Position wird das elementweise Produkt zwischen Kernel und dem überdeckten Bildausschnitt aufsummiert. Formal ergibt sich für ein zweidimensionales Bild f mit Kernel k :

$$g(x, y) = \sum_{i,j} f(x+i, y+j) k(i, j) \quad (5.20)$$

Das Ergebnis ist eine *Feature Map*, die beschreibt, wie stark ein bestimmtes Muster, wie eine Kante oder Textur an jeder Bildposition vorhanden ist [vgl. 52, S. 14]. In Abb. 5.4

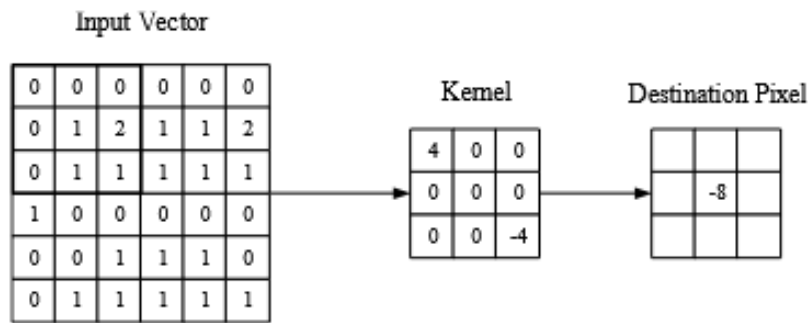


Abb. 5.3: Convolutional [50, S. 6]

wird ein CNN dargestellt. Die Architektur basiert auf drei Schichttypen. In der Faltungsschicht wird, wie der Name es bezeichnet, die Faltung durchgeführt und Merkmale werden extrahiert [vgl. 50, S. 5 f.]. Im Anschluss an die Faltungsoperationen folgt eine Aktivierungsfunktion. Diese fügt eine Nichtlinearität hinzu und reduziert das Vanishing-Gradient-Problem [vgl. 51, S. 1169]. Als Aktivierungsfunktion wird standardmäßig die Rectified Linear Unit (ReLU) mit $f(x) = \max(0, x)$ eingesetzt, die ein schnelleres Training als traditionelle Funktionen wie $\tanh(x)$ ermöglicht [vgl. 53, S. 3].

Darauf folgt die *Pooling-Schicht*. Diese reduziert die räumliche Auflösung, typischerweise durch Max-Pooling mit 2×2 -Fenstern, wodurch die Aktivierungskarte auf 25% ihrer Größe verkleinert wird. Damit werden die Anzahl der Parameter sowie die Rechenkomplexität des Modells schrittweise reduziert [vgl. 50, S. 8].

Die *Fully-Connected-Schicht* am Ende des Netzwerks verbindet jedes Neuron direkt mit allen Neuronen der benachbarten Schichten, analog zur klassischen ANN-Architektur. Dadurch werden die extrahierten Merkmale zu Klassenwahrscheinlichkeiten verarbeitet [vgl. 50, S. 8]. Im Gegensatz zu traditionellen ANN, bei denen jedes Neuron

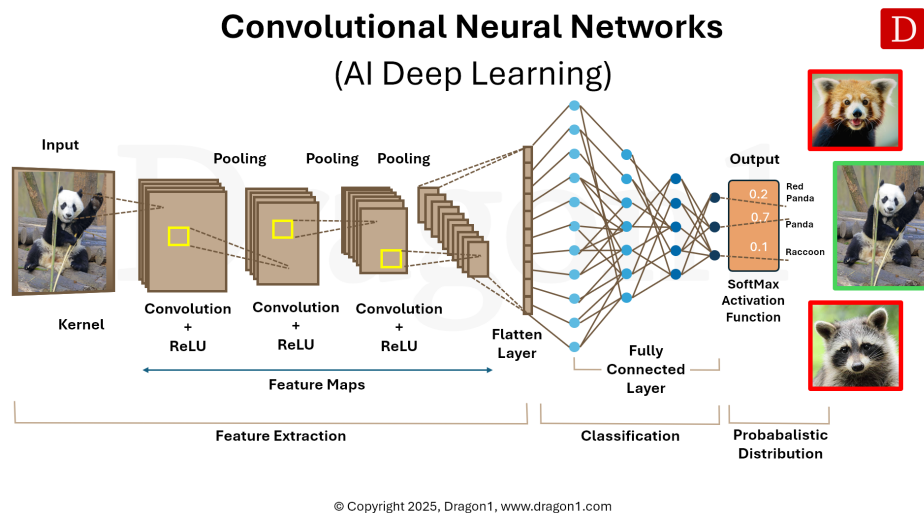


Abb. 5.4: CNN

mit allen Neuronen der vorherigen Schicht verbunden ist, verbinden sich Neuronen in CNN nur mit einem kleinen lokalen Bereich der Eingabe [vgl. 50, S. 6]. Diese lokale Konnektivität reduziert die Anzahl der Parameter erheblich und macht das Training effizienter [vgl. 50, S. 2 f.].

Eine typische CNN-Architektur stapelt mehrere Faltungs- und Pooling-Schichten, wobei frühe Schichten einfache Merkmale und tiefere Schichten zunehmend komplexe, abstrakte Repräsentationen lernen [vgl. 50, S. 8 f.]. Diese hierarchische Merkmalsextraktion bildet die Grundlage für den Encoder-Pfad in Architekturen wie U-Net [vgl. 49, S. 6].

5.4.2 U-Net

U-Net ist eine CNN-Architektur, die ursprünglich für die biomedizinische Bildsegmentierung entwickelt wurde [vgl. 54, S. 2]. Trotz ihres ursprünglichen Anwendungsbereiches dient die Architektur eine zentrale Rolle im Denoising-Schritt in Diffusionsmodellen, in denen sie als Backbone zur Rauschvorhersage eingesetzt wird [vgl. 44, S. 5] [vgl. 49, S. 2]. In Abb. 5.5 wird die Architektur dargestellt. Das Grundprinzip von U-Net basiert auf einer symmetrischen Encoder-Decoder-Struktur. Der Encoder, auch als *Contracting Path* bezeichnet, folgt dem Prinzip eines typischen CNNs und besteht aus wiederholten Blöcken von zwei 3×3 -Faltungen (unpadded) mit anschließender ReLU-Aktivierung sowie

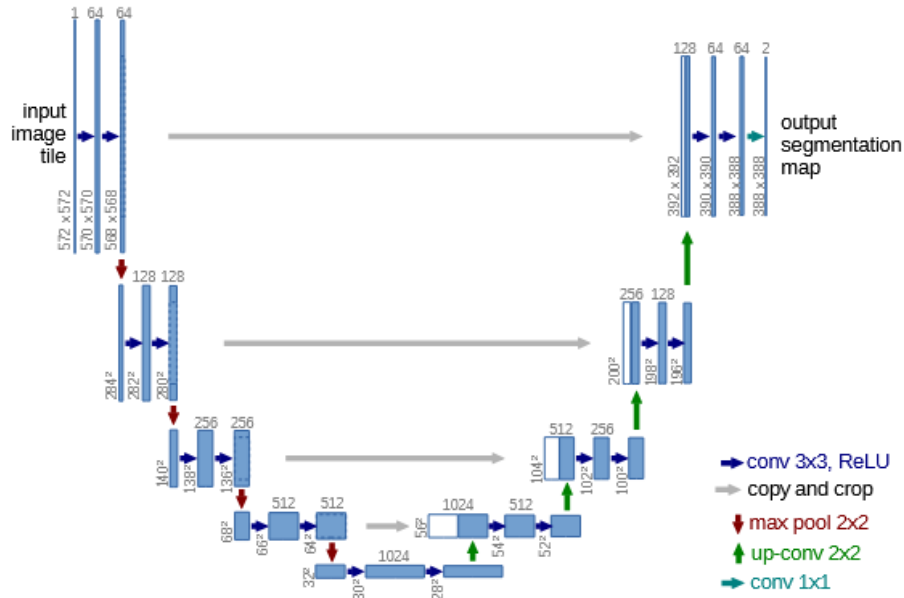


Abb. 5.5: U-Net Architektur [54, S. 2]

einem 2×2 Max-Pooling mit Schrittweite 2 zum Downsampling [vgl. 54, S. 4]. Mit jeder Downsampling-Stufe verdoppelt sich die Anzahl der Feature-Kanäle, während die räumliche Auflösung schrittweise reduziert wird [vgl. 54, S. 4]. Dieser Prozess ermöglicht es dem Netzwerk, zunehmend abstrakte semantische Merkmale zu extrahieren und hierarchische Repräsentationen zu erlernen [vgl. 49, S. 6].

Der Decoder, auch *Expansive Path* genannt, spiegelt diese Struktur symmetrisch und besteht aus Upsampling-Schritten mittels 2×2 -Up-Convolutions, gefolgt von zwei 3×3 -Faltungen mit ReLU [vgl. 54, S. 4]. Das zentrale Merkmal der U-Net-Architektur sind die sogenannten Skip Connections, dargestellt als graue Pfeile in Abb. 5.5. Diese verbinden korrespondierende Encoder- und Decoder-Ebenen direkt miteinander, indem die Feature-Maps konkateniert werden [vgl. 54, S. 4] [vgl. 49, S. 9]. Dadurch können feinkörnige räumliche Informationen, die beim Downsampling verloren gehen, im Decoder wiederhergestellt werden. Die finale Schicht verwendet eine 1×1 -Faltung, um die Feature-Vektoren auf die gewünschte Anzahl von Klassen abzubilden [vgl. 54, S. 4].

Mathematisch lässt sich ein einzelner Faltungsschritt der Ebene l beschreiben als [vgl. 49, S. 9]:

$$x_{l+1} = \sigma(W_l * x_l + b_l), \quad (5.21)$$

wobei W_l den Faltungsfilter, x_l die Eingabe-Feature-Map, b_l den Bias und σ die ReLU-Aktivierungsfunktion bezeichnen. Die Skip Connection im Decoder lässt sich formal als [vgl. 49, S. 9]:

$$F_l = \text{Concat}(E_l, D_l) \quad (5.22)$$

beschreiben, wobei E_l und D_l die Feature-Maps von Encoder und Decoder auf Ebene l repräsentieren und F_l die fusionierte Feature-Map darstellt [vgl. 49, S. 9]. Diese Konkatenation ermöglicht es, hochauflösende strukturelle Details aus dem Encoder mit den abstrakten semantischen Repräsentationen des Decoders zu verbinden.

Im Kontext von Diffusionsmodellen wird U-Net eingesetzt, um bei jedem Denoising-Schritt das Rauschen $\epsilon_\theta(x_t, t)$ zu schätzen, wobei der Zeitschritt t als zusätzliche Konditionierungsinformation in die Architektur einfließt [vgl. 44, S. 5]. Spätere Arbeiten haben die U-Net-Architektur für Diffusionsmodelle weiterentwickelt, unter anderem durch zusätzliche Attention-Schichten und tiefere Encoder-Decoder-Pfade, um die Bildqualität weiter zu steigern [vgl. 29, S. 3 f.]. U-Net lässt sich in verschiedene Domänen und in unterschiedliche Modelltypen integrieren. Durch die Flexibilität der symmetrischen Encoder-Decoder-Struktur mit Skip Connections hat sich U-Net als Standardarchitektur in modernen Diffusionsmodellen für verschiedene Datenmodalitäten etabliert, darunter Bild-, Audio-, Video- und 3D-Generierung [vgl. 49, S. 2].

Die U-Net-Architektur bietet mehrere Vorteile für generative Aufgaben. Die hierarchische Encoder-Decoder-Struktur ermöglicht eine Merkmalsextraktion auf mehreren Auflösungsebenen, wodurch sowohl globaler Kontext als auch lokale Details erfasst werden können. Die Skip Connections behalten detaillierte räumliche, temporale und strukturelle Details, die bei herkömmlichen Autoencoder-Architekturen durch das Downsampling verloren gehen würden [vgl. 54, S. 4]. Darüber hinaus besitzt U-Net eine hohe Recheneffizienz, da die faltungsbasierten Operationen parallelisiert werden können und im Gegensatz zu Transformer-Modellen linear mit der Eingabegröße skalieren. Weitere Stärken umfassen die Robustheit gegenüber verrauschten oder unvollständigen Eingabedaten sowie die Flexibilität zur Integration von Attention-Mechanismen und Residual Connections [vgl. 49, S. 34 ff.].

Trotz dieser Vorteile weist U-Net auch Limitationen auf. Eine wesentliche Einschränkung ist die begrenzte Fähigkeit zur Erfassung globalen Kontexts, da die faltungsbasierte Struktur primär auf lokale räumliche Merkmale spezialisiert ist. Bei hochauflösenden generativen Aufgaben steigt der Speicherbedarf erheblich, während das rezeptive Feld der Faltungsschichten proportional kleiner wird. Zudem kann die Generalisierungsfähigkeit bei domänenfremden Daten oder multimodalen Aufgaben wie Text-zu-Bild-Generierung eingeschränkt sein, da U-Net auf pixelbasierte Transformationen ausgelegt ist. Schließlich operiert U-Net wie viele tiefe neuronale Architekturen als Blackbox, was die Interpretierbarkeit der gelernten Repräsentationen erschwert [vgl. 49, S. 32 f.] [vgl. 49, S. 40 f.].

6 Datenqualität und ethische Aspekte

Das Konzept der Generierung synthetischer Daten lässt sich auf die Arbeiten von Metropolis und Ulam im Jahr 1949 zurückführen, die mit der Entwicklung der Monte-Carlo-Simulationsmethoden den Grundstein für die rechnergestützte Erzeugung künstlicher Datenpunkte legten [vgl. 55, S. 335 f.]. Seitdem haben sich die Methoden zur synthetischen Datenerzeugung weiterentwickelt und finden heute in verschiedenen Anwendungsbereichen Einsatz, wovon eines davon in der Medizin ist. In diesem Kapitel werden die Aspekte der Datenqualität sowie die ethischen Herausforderungen behandelt, die mit der Nutzung synthetischer Daten einhergehen.

6.1 Definition synthetischer Daten

Bevor auf die Erzeugung und Anwendung synthetischer Daten eingegangen wird, ist zunächst eine Definition nötig, um zu verstehen was synthetische Daten sind. Jordon u. a. von der Royal Society und dem Alan Turing Institute definieren den Begriff wie folgt:

„Synthetic data is data that has been generated using a purpose-built mathematical model or algorithm, with the aim of solving a (set of) data science task(s).“ [56, S. 5]

Aus dieser Definition lassen sich zwei Aspekte ableiten: Zum einen werden synthetische Daten nicht zufällig erzeugt, sondern gezielt durch mathematische Modelle oder Algorithmen generiert. Darunter fallen beispielsweise GAN, VAE oder Diffusionsmodelle, die jeweils unterschiedliche Strategien zur Datenerzeugung verfolgen. Zum anderen sind synthetische Daten immer zweckgebunden. Das heißt, sie werden gezielt eingesetzt, um datengetriebene Aufgaben wie Klassifikation, Vorhersage oder den Schutz sensibler Informationen zu lösen.

Unabhängig von der Erzeugungsmethode lassen sich synthetische Daten in partiell und vollständig synthetische Daten unterteilen [vgl. 57, S. 2]. Partiiell synthetische Daten kombinieren reale Datensätze mit synthetisch erzeugten Elementen, wobei gezielt Variablen mit hohem Offenlegungsrisiko ersetzt werden, während die restlichen Merkmale erhalten bleiben. Im medizinischen Bereich wird dieser Ansatz etwa genutzt, um die Privatsphäre von Patientinnen und Patienten zu schützen, um dennoch aussagekräftige Analysen zu erlauben [vgl. 58, S. 7]. Vollständig synthetische Daten hingegen werden komplett durch statistische Modelle erzeugt, ohne dass die Originaldaten preisgegeben werden. Dabei kommt das Verfahren der multiplen Imputation zum Einsatz, bei dem alle nicht erhobenen Werte als fehlend betrachtet und wiederholt durch plausible Werte ersetzt werden [vgl. 59, S. 2]. Beide Ansätze können dazu beitragen, Probleme wie Datenknappheit oder Datenschutzerfordernngen anzugehen. Mehr dazu in Abschnitt 6.2.

6.2 Ethische Aspekte synthetischer medizinischer Daten

Obwohl synthetische Daten von rechnergestützten Systemen generiert werden und keine unmittelbaren Patienteninformationen enthalten, ergeben sich durch ihre Erzeugung und Nutzung im medizinischen Umfeld ethische Herausforderungen. Da generative Modelle auf realen Patientendaten trainiert werden, können sich Probleme des Datenschutzes und der Verantwortung auch auf den synthetischen Datenraum übertragen.

Medizinische Bilddaten zählen zu den schützenswerten personenbezogenen Daten. Art. 9 Abs. 1 DSGVO legt fest, dass die Verarbeitung von „Gesundheitsdaten [...] einer natürlichen Person [...] untersagt“ ist. Von diesem Verarbeitungsverbot darf nur unter engen Voraussetzungen, wie einer ausdrücklichen Einwilligung ausgewichen werden [60]. Dies schränkt die Entwicklung KI-basierter Verfahren in der medizinischen Bildverarbeitung ein, da der Zugang zu Trainingsdaten begrenzt ist. Synthetische Daten bieten einen Lösungsansatz, da sie keine direkten Bezüge zu realen Personen enthalten und daher außerhalb des Anwendungsbereichs der DSGVO liegen.

Ergänzend stellt die KI-Verordnung Anforderungen an die Datenbasis KI-basierter Anwendungen in der Medizin, die nach Art. 6 Abs. 1 AI Act als Hochrisiko-KI-Systeme gelten [61]. So fordert Art. 10 AI Act, dass Trainings-, Validierungs- und Testdatensätze „relevant, hinreichend repräsentativ und so weit wie möglich fehlerfrei und vollständig“ sind [62]. Besonders die Forderung nach Repräsentativität ist aus ethischer Sicht von Bedeutung, da sie für eine gerechte und diskriminierungsfreie Anwendung medizinischer KI-Systeme bildet. Hieraus ergibt sich das Problem, dass die Verzerrungen (auch *Bias* genannt) in den Daten vermieden werden müssen. Denn es besteht bei der Generierung synthetischer Daten das Risiko, dass identifizierbare Merkmale erhalten bleiben und sensible Informationen indirekt preisgegeben werden. Zwar können sensible Daten vor dem Training anonymisiert werden, eine zu starke Anonymisierung verringert jedoch den analytischen Wert der Daten [vgl. 63, S. 192]. Somit besteht ein Kompromiss zwischen Anonymität und Wert der Daten.

6.3 Risiken und Grenzen synthetischer Daten

Neben den ethischen Fragestellungen weisen synthetische Daten auch technische und methodische Grenzen auf. Eine Herausforderung liegt in der medizinischen Plausibilität synthetischer Daten. Generative Modelle können visuell gute Ergebnisse liefern, dabei jedoch anatomisch fehlerhafte oder medizinisch unrealistische Strukturen erzeugen. Diese Daten werden als Halluzinationen bezeichnet. Diese Halluzinationen führen dazu, dass die synthetischen Bilder bei der Auswertung irreführende Ergebnisse liefern und somit die Zuverlässigkeit des KI-Modells verringert [vgl. 64, S. 10]. Hinzu kommt, dass GANs häufig unter Mode Collapse leiden, wodurch die Vielfalt der generierten Bilder eingeschränkt wird. Jedoch wird dieses Problem durch die Verwendung von beispielsweise Diffusions Modellen verringert [vgl. 64, S. 5] [vgl. 36, S. 1].

Dazu zählt aber jedoch das Problem der Generalisierbarkeit. Modelle für medizinische Bilddaten lernen die Muster ihrer Trainingsdaten. Jedoch kann diese Fähigkeit aber nicht zwangsläufig auf neue, abweichende Bildmuster übertragen. Beispielsweise ein

Klassifikationsmodell für die Hautläsion, das überwiegend mit Bildern heller Hauttypen trainiert wurde, liefert bei dunkleren Hauttypen schlechtere Ergebnisse [vgl. 65, S. 18]. Da generative Modelle die Eigenschaften ihrer Trainingsdaten übernehmen, werden solche Verzerrungen auch in synthetischen Daten weitergegeben. Hinzu kommt, dass generative Modelle für ihr initiales Training selbst auf reale klinische Bilddaten angewiesen sind [vgl. 64, S. 10]. Synthetische Daten bieten somit nur eine Teillösung für den Mangel an medizinischen Trainingsdaten und können reale Patientendaten nicht komplett ersetzen.

6.4 Qualitätskriterien für synthetische medizinische Bilddaten

Um die Eignung synthetischer Bilddaten für medizinische Anwendungen beurteilen zu können, sind verlässliche Qualitätskriterien erforderlich. Diese lassen sich in visuelle und mathematische Bewertungsverfahren unterteilen, die im Folgenden näher beschrieben werden.

6.4.1 Visuelle Bewertungskriterien

Die visuelle Bewertung synthetischer medizinischer Bilder erfolgt im besten Fall durch medizinische Fachkräfte. Diese Form der Beurteilung ist deshalb relevant, weil rein mathematische Metriken medizinisches Fachwissen nicht abbilden können und daher nicht zur Beurteilung der medizinischen Plausibilität geeignet sind [vgl. 66, S. 6] [vgl. 67, S. 6].

In einer Studie von Schuit, Parra und Besa wurde eine solche Reader Study mit drei Radiologen durchgeführt. Die Teilnehmer sollten sowohl zwischen realen und synthetischen Röntgenaufnahmen unterscheiden als auch die Konsistenz zwischen Bildmerkmalen und Zielpathologie bewerten. Die Ergebnisse zeigen, dass insbesondere Diffusionsmodelle visuell realistische Röntgenaufnahmen erzeugen können [vgl. 67, S. 1].

Dennoch weist die visuelle Bewertung Einschränkungen auf. Sie ist subjektiv und von der Erfahrung der beurteilenden Person abhängig. Zudem lässt sie sich schwer auf große Datenmengen skalieren, weshalb die Bewertung durch mathematische Metriken ergänzt werden kann.

6.4.2 Mathematische Bewertungsmetriken

Um die Qualität von durch generativen Modelle erstellten Bildern objektiv zu beurteilen, kommen sogenannte *Image Quality Metrics* (IQMs) zum Einsatz. IQMs erlauben einen objektiven Rahmen zur Bewertung von Eigenschaften wie Auflösung, Rauschen und Artefakte. Gegenüber subjektiven Bewertungen durch Menschen haben IQMs den Vorteil, dass sie standardisiert, quantifizierbar und reproduzierbar sind und somit einen systematischen Vergleich verschiedener generativer Modelle ermöglicht. IQMs lassen sich in drei verschiedenen Kategorien einordnen: full-reference (FR), reduced-reference (RR) und

no-reference (NR). Diese Unterscheidung ist im Kontext von Diffusionsmodellen besonders wichtig. Bei Aufgaben wie Bildrekonstruktion oder Denoising, bei denen ein Originalbild als Referenz vorliegt, kommen FR-Metriken wie SSIM oder PSNR zum Einsatz. Bei der freien Text-to-Image-Generierung hingegen existiert kein eindeutiges Referenzbild, weshalb hier häufig auf NR-Metriken oder semantische Maße wie den FID oder den CLIP-Score zurückgegriffen wird [vgl. 68, S. 1]. **TODO: mehr Quellen zu IQM finden**

Fréchet Inception Distance

Die *Fréchet Inception Distance* (FID) ist eine Metrik zur Bewertung der Qualität GAN und wurde 2017 von Heusel et al. eingeführt. Sie stellt eine Weiterentwicklung des Inception score (IS) dar, dessen Schwachpunkt darin besteht, dass er die statistische Verteilung realer Bilder nicht in die Bewertung einbezieht [vgl. 69, S. A1]. Die Berechnung der FID erfolgt in zwei Schritten. Zunächst werden sowohl reale als auch generierte Bilder durch ein vortrainiertes CNN geleitet [vgl. 70, S. 2]. Aus einer intermediären Schicht dieses Netzwerks werden abstrakte Merkmalsvektoren gewonnen, die als visuelle Repräsentationen der Bilder dienen [vgl. 71, S. 130]. Im zweiten Schritt werden diese Merkmale statistisch beschrieben. Für die realen Bilder wird der Mittelwert μ_{data} also der durchschnittliche Merkmalsvektor sowie die Kovarianzmatrix Σ_{data} berechnet, welche beschreibt, wie stark die Merkmale untereinander variieren. Dasselbe geschieht für die generierten Bilder, die durch μ_g und Σ_g beschrieben werden [vgl. 71, S. 130] [vgl. 69, S. A1]. Anschließend wird die Fréchet Distanz zwischen den beiden so beschriebenen Verteilungen berechnet [vgl. 72, S. 2]:

$$FID = \|\mu_{data} - \mu_g\|^2 + \text{Tr}\left(\Sigma_{data} + \Sigma_g - 2\left(\Sigma_{data}\Sigma_g\right)^{1/2}\right), \quad (6.1)$$

Die Formel Gl. (6.1) besteht aus zwei Teilen. Der erste Term $\|\mu_{data} - \mu_g\|^2$ misst, wie weit die durchschnittlichen Merkmale realer und generierter Bilder voneinander abweichen. Der zweite Term vergleicht die Kovarianzmatrizen beider Verteilungen und erfasst damit, ob die generierten Bilder eine ähnliche Vielfalt und Struktur aufweisen wie die realen cite [vgl. 72, S. 2]. In Abb. 6.1 wird der Prozess dargestellt. Der Generator G erzeugt Bilder,

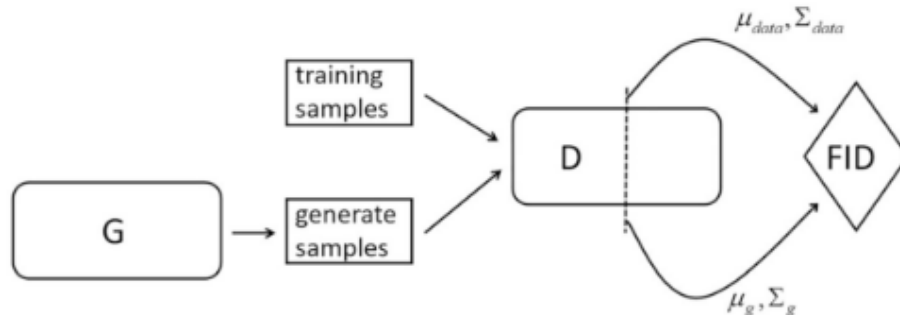


Abb. 6.1: Darstellung der FID-Berechnung [71, S. 130]

die zusammen mit den Trainingsbildern durch das CNN D geleitet werden, um die statistischen Kenngrößen μ_{data} , Σ_{data} , μ_g und Σ_g zu extrahieren und daraus den FID-Wert

zu berechnen. Ein niedriger FID-Wert zeigt an, dass beide Verteilungen eng beieinander liegen, was auf hohe Bildqualität und ausreichende Diversität hindeutet [vgl. 71, S. 130].

Structural Similarity Index Measure

Der *Structural Similarity Index Measure* (SSIM) ist eine Metrik zur Bewertung von Bildqualität, die sich an der menschlichen Wahrnehmung orientiert [vgl. 73, S. 1131] [vgl. 68, S. 6]. Im Gegensatz zur Mean squared Error (MSE), der lediglich pixelweise Helligkeitsunterschiede misst, analysiert der SSIM, wie ähnlich zwei Bilder in ihrer Struktur sind also in den Mustern und Abhängigkeiten benachbarter Pixel, die für das menschliche Auge relevant sind [vgl. 74, S. 1]. Dazu zerlegt SSIM den Bildvergleich in drei Teilkomponenten: Intensität, Kontrast und Struktur. Diese werden getrennt berechnet und anschließend multipliziert [vgl. 74, S. 5 f.]. Die resultierende Formel lautet [vgl. 75, S. 2] [vgl. 74, S. 5] [vgl. 68, S. 6] [vgl. 73, S. 1134]:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (6.2)$$

In Gl. (6.2) stehen μ_x und μ_y für die mittleren Helligkeitswerte, σ_x^2 und σ_y^2 für die lokalen Varianzen und σ_{xy} für die Kovarianz also das Maß, wie ähnlich sich die Helligkeitsverläufe der beiden Bilder in einem lokalen Bereich verhalten. Die Konstanten C_1 und C_2 verhindern numerische Instabilitäten bei sehr kleinen Nennerwerten und sind als $C_1 = (K_1 L)^2$ und $C_2 = (K_2 L)^2$ definiert, wobei L den Wertebereich der Pixel angibt und typischerweise $K_1 = 0,01$ sowie $3K_2 = 0,03$ gewählt werden [vgl. 75, S. 1 f.] [vgl. 74, S. 5] [vgl. 68, S. 6] [vgl. 73, S. 1134]. Der berechnete SSIM-Wert eines Bildes ergibt sich als Mittelwert aller lokal berechneten Fensterwerte und liegt stets zwischen 0 und 1, wobei 1 für eine vollständige Übereinstimmung bedeutet [73, S. 1131]. Jedoch ist der SSIM eine FR-Metrik und benötigt somit ein fehlerfreies Referenzbild [74, S. 1]

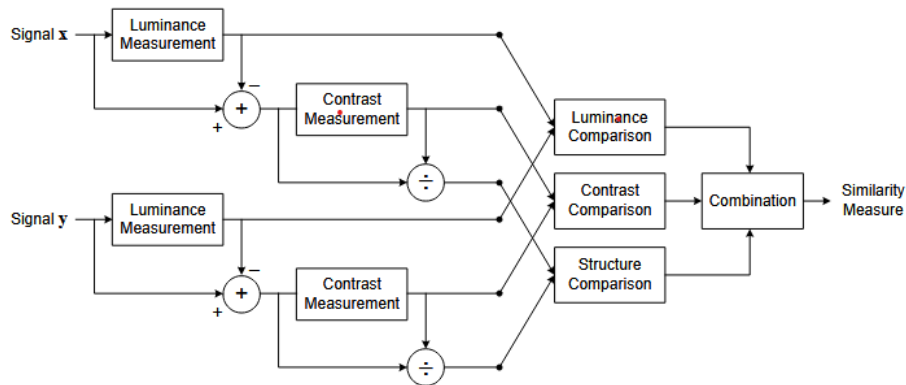


Abb. 6.2: Darstellung der SSIM-Berechnung [74, S. 5]

Peak Signal-to-Noise Ratio

Die *Peak Signal-to-Noise Ratio* (PSNR) bewertet die Bildqualität, indem sie das generierte Bild pixelweise mit dem Referenzbild vergleicht. Sie wird über den mittleren quadrati-

schen Fehler MSE zwischen beiden Bildern bestimmt und in Dezibel angegeben [vgl. 76, S. 3]:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right) \quad (6.3)$$

Dabei ist MAX der maximal mögliche Pixelwert (z. B. 255 bei 8-Bit-Bildern oder 1 bei normalisierten Bildern) und MSE der mittlere quadratische Fehler zwischen Referenz- und generiertem Bild.

Ein hoher PSNR-Wert deutet auf eine hohe Bildqualität hin. Je kleiner die Differenz zwischen den beiden Bildern, desto größer der Wert [vgl. 71, S. 137]. Allerdings bildet die PSNR die menschliche Wahrnehmung nicht zuverlässig ab, da sie ausschließlich auf Pixeldifferenzen beruht. Bereits eine kleine räumliche Verschiebung der Pixel kann zu einem schlechteren PSNR-Wert führen, obwohl die subjektiv wahrgenommene Bildqualität unverändert bleibt. Aus demselben Grund erreichen Modelle mit pixelbasiertem Trainingsziel typischerweise höhere PSNR-Werte als generative Modelle [vgl. 76, S. 3]. Die PSNR ist daher zwar ein etabliertes, aber kein hinreichendes Qualitätsmaß und sollte im Kontext generativer Modelle durch wahrnehmungsbasierte Metriken wie LPIPS oder FID ergänzt werden.

Learned Perceptual Image Patch Similarity

Die *Learned Perceptual Image Patch Similarity* (LPIPS) ist eine FR-Metrik. Sie misst die Ähnlichkeit zweier Bilder nicht im Pixelraum wie SSIM oder PSNR, sondern anhand der Merkmale, die ein tiefes neuronales Netz aus den Bildern extrahiert. Sie wurde 2018 von Zhang u. a. eingeführt, weil pixelbasierte Maße die wahrgenommene Ähnlichkeit zweier Bilder oft schlecht widerspiegeln, insbesondere bei Verzerrungen wie Unschärfen oder Verschiebungen [vgl. 77, S. 1].

Zur Berechnung werden beide Bilder x und x_0 durch ein vortrainiertes CNN (z. B. VGG oder AlexNet) geleitet. Aus L Schichten werden die Merkmalskarten $\phi_l(x)$ und $\phi_l(x_0)$ entnommen und kanalweise normalisiert. Anschließend wird an jeder Position die ℓ_2 -Distanz berechnet, über die Fläche der Schicht gemittelt, mit einem gelernten Gewicht w_l skaliert und über alle Schichten summiert [vgl. 77, S. 5 f.]:

$$\text{LPIPS}(x, x_0) = \sum_{l=1}^L \frac{w_l}{H_l W_l} \sum_{h,w} \|\phi_l(x)_{h,w} - \phi_l(x_0)_{h,w}\|_2^2 \quad (6.4)$$

In Gl. (6.4) steht $\phi_l(x)_{h,w}$ für den normalisierten Merkmalsvektor an Position (h, w) in Schicht l mit räumlichen Dimensionen $H_l \times W_l$, und w_l ist das gelernte Gewicht für diese Schicht [77, S. 6]. Die Gewichte w_l werden auf dem Berkeley-Adobe Perceptual Patch Similarity (BAPPS)-Datensatz mit rund 484 000 menschlichen Bewertungen aus 2AFC-Tests trainiert, sodass jene Schichten stärker beitragen, die mit der menschlichen Wahrnehmung übereinstimmen [vgl. 77, S. 2 f.].

Ein niedriger LPIPS-Wert bedeutet hohe wahrgenommene Ähnlichkeit. Im Vergleich zu klassischen Metriken wie ℓ_2 , SSIM oder FSIM stimmt LPIPS deutlich stärker mit menschlichen Urteilen überein, benötigt aber wie andere FR-Metriken ein Referenzbild [vgl. 77, S. 7].

Dice Similarity Coefficient

Der *Dice Similarity Coefficient* (DSC) ist eine Überlappungsmetrik aus der Bildsegmentierung. Er wurde ursprünglich 1945 von Dice als *coincidence index* für ökologische Anwendungen eingeführt [vgl. 78, S. 298]. Jedoch hat es sich später in der medizinischen Bildverarbeitung als Standardmaß zur Bewertung von Segmentierungen etabliert [vgl. 79, S. 2]. Er misst die räumliche Überlappung zweier Segmentierungen, einer erzeugten und einer als Referenz dienenden Segmentierung. Für zwei Mengen A und B , die die jeweiligen segmentierten Bereiche darstellen, ist er definiert als [vgl. 79, S. 4]:

$$\text{DSC}(A, B) = \frac{2|A \cap B|}{|A| + |B|} \quad (6.5)$$

Der Zähler entspricht der doppelten Anzahl der gemeinsamen Voxel, der Nenner der Summe der Voxel beider Mengen. Der Wertebereich liegt zwischen 0 und 1, wobei 0 keine und 1 vollständige Überlappung bedeutet; Werte über 0,7 gelten in der Praxis als gute Übereinstimmung [vgl. 79, S. 4].

7 Implementierung und experimenteller Aufbau

Das folgende Kapitel beschreibt die praktische Umsetzung. Zunächst werden die Projektmethodik und die eingesetzten Werkzeuge vorgestellt, anschließend Datenvorverarbeitung, Modellarchitektur und Trainingskonfiguration. Auf die Ablationsstudien folgen die conditional Modelle samt Generierungs-Pipeline sowie die abschließende Auswertung der erzeugten Volumina.

7.1 Projektmethodik

Das Projekt wurde nach einem Kanban-orientierten Vorgehen organisiert. Diese Methode bietet sich für Arbeiten mit experimentellem Charakter an, bei denen sich der nächste Arbeitsschritt erst aus den Ergebnissen des vorherigen ergibt.

Die einzelnen Aufgaben wie das Training der Baseline, die Ablationsvarianten und die conditional Modelle wurden in kleinere Teilaufgaben unterteilt und aus einer Bearbeitungsliste abgearbeitet. Die Reihenfolge der Aufgaben ergab sich laufend aus den Ergebnissen der vorhergehenden Trainingsläufe und wurde nicht vorab festgelegt.

7.2 Entwicklungsumgebung und Hardware

Das Training der Modelle wurde auf einem lokalen System durchgeführt. Dieses verfügt über eine NVIDIA GeForce GTX 1080 Ti mit 11 GB VRAM, einen AMD Athlon X4 860K Prozessor (4 Kerne, 3,7 GHz), 16 GB Arbeitsspeicher sowie ein Linux-Ubuntu-Desktop-Betriebssystem. Der Einsatz einer dedizierten GPU ermöglicht eine Reduktion der Trainingszeiten gegenüber einer reinen CPU-basierten Berechnung.

Die Implementierung sämtlicher Modelle erfolgte in der Programmiersprache Python. Python besitzt eine umfangreiche Anzahl an Bibliotheken, wie Keras und TensorFlow für maschinelles Lernen und hat sich als De-facto-Standard in diesem Bereich etabliert [80]. Als Entwicklungsumgebung wurde Visual Studio Code in Kombination mit Jupyter-Notebooks eingesetzt. Die Verwendung von Jupyter-Notebooks ermöglicht eine schrittweise Ausführung einzelner Code-Zellen, was bei der Entwicklung und dem Debugging von Vorteil ist. Für jedes Modell wurde ein separates Notebook angelegt, um eine klare Trennung der Experimente zu gewährleisten.

Für den Modellaufbau wurde das Deep-Learning-Framework PyTorch eingesetzt. Die Diffusionslogik einschließlich des Noise Schedulers, des Sampling-Verfahrens und der Trainingsschleife wurde auf Basis von PyTorch implementiert. Für das Laden und

Speichern der medizinischen Bilddaten im NIfTI-Format wurde die Bibliothek nibabel verwendet. Die Visualisierung der Ergebnisse erfolgte mit Matplotlib. Die Implementierung basiert auf Python 3.12.3 und PyTorch 2.7.1 mit CUDA 11.8. Die Wahl der CUDA-Version ergibt sich aus der Tatsache, dass die NVIDIA GeForce GTX 1080 Ti auf der Pascal-Architektur (Compute Capability 6.1) basiert, welche von neueren CUDA-Versionen nicht mehr vollständig unterstützt wird.

7.3 Dataset und Datenvorverarbeitung

Als Datengrundlage dient der öffentlich verfügbare Datensatz des Brain Tumor Segmentation (BraTS) Challenge 2023 [81]. Der Trainingsdatensatz umfasst 1251 multimodale MRT-Untersuchungen von Patienten mit Hirntumoren. Die Aufnahmen stammen aus mehreren Kliniken. Für jeden Datensatz liegen vier MRT-Sequenzen vor: T1-gewichtete (T1N), kontrastverstärkte T1-gewichtete (T1C), T2-gewichtete (T2W) sowie eine T2-FLAIR-Aufnahme. Dabei wurden alle Volumina bereits vorverarbeitet. Sie wurden räumlich ausgerichtet und auf das Gehirn freigestellt (skull-stripping). Zusätzlich enthält jeder Datensatz eine Tumor-Segmentierungsmaske. Für die Hausarbeit werden ausschließlich die nativen T1-gewichteten (T1N) Volumina sowie die zugehörigen Segmentierungsmasken verwendet. Die unconditional Modelle werden allein auf den T1N-Volumina trainiert, während die conditional Modelle zusätzlich auf die Segmentierungsmasken zurückgreifen.

Vor dem Training wurden die MRT-Volumina des BraTS-2023-Datensatzes vorverarbeitet. Die Vorverarbeitung orientiert sich an der Datenverarbeitung von Khader u. a. [36, S. 4]. Es umfasst fünf Schritte, die im Folgenden beschrieben werden.

Intensitätsbegrenzung Die Intensitätswerte werden auf das Intervall zwischen dem 0,5%- und dem 99,5%-Perzentil der Vordergrundvoxel beschnitten. Dadurch werden extreme Ausreißer entfernt, die etwa durch Aufnahmeartefakte entstehen können, ohne relevante Bildinformationen zu verlieren.

Hintergrundentfernung Anschließend wird der Hintergrund entfernt, der hauptsächlich aus Luft und leeren Voxeln besteht. Dazu wird die kleinste Bounding Box berechnet, die alle Voxel mit einem Intensitätswert größer null enthält, erweitert um einen Rand von zwei Voxeln je Achse. Das Volumen wird so auf den anatomisch relevanten Bereich zugeschnitten.

Kubisches Padding Da die Volumina nach dem Cropping unterschiedliche Dimensionen aufweisen, werden sie durch symmetrisches Zero-Padding auf eine kubische Form gebracht. Im Gegensatz zu einer direkten Skalierung bleibt durch das Padding das Seitenverhältnis erhalten und es werden keine geometrischen Verzerrungen eingeführt.

Resizing Das kubische Volumen wird mittels trilinearer Interpolation auf die Zielauflösung von 32^3 oder 48^3 Voxeln skaliert. Da das Volumen bereits kubisch ist, erfolgt die Skalierung in allen drei Raumrichtungen gleichmäßig.

Normalisierung Abschließend werden die Intensitätswerte je Volumen per Min-Max-Skalierung auf den Wertebereich $[-1, 1]$ abgebildet. Dies ist die gängige Konvention für Diffusionsmodelle, da der Rauschprozess auf einem standardisierten Wertebereich arbeitet.

Segmentierungsmasken Die Tumor-Segmentierungsmasken durchlaufen eine reduzierte Variante derselben Pipeline. Die Vordergrund-Bounding-Box wird nicht aus der Maske selbst, sondern aus dem zugehörigen T1N-Volumen übernommen, damit Bild und Maske räumlich übereinander liegen. Auf die Intensitätsbegrenzung und die Normalisierung wird verzichtet, da die Maske diskrete Klassenlabels enthält. Das Resizing erfolgt entsprechend per Nearest-Neighbor-Interpolation, sodass die Labelwerte erhalten bleiben. Die vier Labelwerte des BraTS-2023-Schemas bezeichnen Hintergrund (0), nekrotischen Tumorkern (1), peritumorales Ödem (2) und anreichernden Tumor (3).

7.4 Modellarchitektur

Als Architektur dient ein 3D-UNet, das dem in der DDPM-Literatur etablierten Aufbau folgt Ho, Jain und Abbeel. Die Architektur ist symmetrisch als Encoder-Decoder aufgebaut und besteht aus vier Auflösungsstufen mit Kanalbreiten von 96, 192, 384 und 384. Im Encoder wird die räumliche Auflösung schrittweise halbiert, während der Decoder sie über trilineare Interpolation wieder auf die Eingabegröße bringt. Zwischen den entsprechenden Stufen von Encoder und Decoder werden Skip-Connections eingesetzt, um räumliche Informationen direkt weiterzugeben.

Residualblöcke und Zeiteinbettung. Auf jeder Auflösungsstufe befinden sich zwei Residualblöcke, die jeweils aus zwei 3D-Faltungsschichten mit GroupNorm (acht Gruppen) und SiLU-Aktivierung¹ bestehen. Der aktuelle Diffusions-Zeitschritt wird zunächst sinusoidal kodiert und durch ein zweischichtiges MLP auf 384 Dimensionen (das Vierfache der Basis-Kanalbreite) gebracht. In jedem Residualblock wird diese Zeit-Einbettung anschließend über eine lineare Projektion auf die jeweilige Kanalzahl skaliert und additiv eingespeist, sodass das Netzwerk sein Verhalten an den jeweiligen Rauschgrad anpassen kann.

¹Die SiLU-Funktion ist definiert als $\text{SiLU}(x) = x \cdot \sigma(x)$ mit der Sigmoid-Funktion σ . Im Gegensatz zum stückweise linearen ReLU verläuft sie glatt und differenzierbar und hat sich in tiefen Generativmodellen etabliert [vgl. 82, S. 4].

Self-Attention. Auf den beiden niedrigsten Auflösungsstufen werden zusätzlich Self-Attention-Schichten eingesetzt, die es dem Netzwerk ermöglichen, globale Abhängigkeiten zwischen räumlich entfernten Regionen zu modellieren. Auf höheren Auflösungen wird darauf verzichtet, da der Speicherbedarf mit der Voxelanzahl ansteigt. Bei einer Eingabegröße von 32^3 Voxeln betrifft dies die Auflösungen 8^3 und 4^3 , bei 48^3 entsprechend 12^3 und 6^3 .

Im Bottleneck zwischen Encoder und Decoder befindet sich eine weitere Self-Attention-Schicht. Diese ist fest in der Architektur und bleibt auch in der Ablation ohne Attention auf den niedrigen Stufen erhalten.

Initialisierung. Die finale 3D-Faltung am Ausgang des Netzwerks wird mit Nullen initialisiert. Das Modell liefert dadurch zu Trainingsbeginn eine konstante Nullvorhersage. Erst durch die Gradienten der ersten Iterationen formen sich von Null verschiedene Ausgaben. Diese Initialisierung stabilisiert den Verlust zu Beginn des Trainings und ist in der Diffusions-Literatur eine gängige Wahl.

7.5 Baseline-Konfiguration

Die Baseline-Konfiguration benutzt mehrere Techniken die sich in der aktuellen Literatur als Verbesserungen gegenüber dem ursprünglichen DDPM-Ansatz von Ho, Jain und Abbeel etabliert haben. Sie dient als Referenzmodell, gegen das alle weiteren Experimente verglichen werden.

Der Diffusionsprozess verwendet 250 Zeitschritte mit einem Cosine Schedule empfohlen von Nichol und Dhariwal. Im Vergleich zum linearen Schedule verteilt der Cosine Schedule das Signal-Rausch-Verhältnis gleichmäßiger über die Zeitschritte was insbesondere bei niedrigen Auflösungen zu einer höheren Bildqualität führt [vgl. 45, S. 7].

Das Modell wird mit v -Prediction trainiert, vorgeschlagen von Salimans und Ho. Dabei sagt das Netzwerk weder das Rauschen ϵ noch das ursprüngliche Bild x_0 direkt vorher, sondern eine Kombination aus beiden, die als Velocity v bezeichnet wird. Dieses Vorhersageziel hat sich als numerisch stabiler erwiesen, insbesondere bei niedrigen Signal-Rausch-Verhältnissen [vgl. 83, S. 5].

Für die Gewichtung des Trainingsverlusts wird das von Hang u. a. vorgeschlagene Min-SNR Weighting mit $\gamma = 5$ eingesetzt. Diese Strategie begrenzt den Einfluss von Zeitschritten mit hohem Signal-Rausch-Verhältnis auf den Gesamtverlust und sorgt so für ein ausgewogeneres Training über alle Zeitschritte hinweg [vgl. 84, S. 4 f.].

Während des Trainings werden die Zeitschritte nicht uniform gezogen, sondern über ein loss-bewusstes Importance Sampling auf Basis der jüngsten Verlust-Historie [vgl. 45, S. 5]. Pro Zeitschritt wird der quadratische Mittelwert der Verluste der letzten zehn Batches geführt und daraus eine Wahrscheinlichkeitsverteilung gebildet, sodass Zeitschritte mit hohen Verlusten häufiger angesteuert werden. Damit der Erwartungswert des Trainings-Loss unverändert bleibt, wird jeder Sample-Verlust mit dem Inversen seiner Ziehungswahrscheinlichkeit gewichtet. Die Strategie wird erst nach dem Warmup aktiv, damit instabile Initialverluste die Sampler-Historie nicht verzerren.

7.6 Trainingsverfahren und Hyperparameter

Alle Modelle werden über 300 Epochen mit dem AdamW-Optimizer (Weight Decay 0,01) und einer initialen Lernrate von 2×10^{-4} trainiert. Dieser Wert entspricht dem in der DDPM-Literatur üblichen Lernratenniveau [vgl. 44, S. 14]. In den ersten 500 Iterationen wird die Lernrate linear von null auf den Zielwert erhöht (Warmup), um Instabilitäten zu Beginn des Trainings zu vermeiden, und fällt anschließend nach einem Cosine-Schedule wieder auf null ab. Die Batch Size beträgt 10, was dem verfügbaren GPU-Speicher von 11 GB geschuldet ist.

Zur Stabilisierung der Modellgewichte wird Exponential Moving Average (EMA) mit einem Decay-Faktor von 0,9999 eingesetzt [vgl. 44, S. 15]. Dabei wird parallel zu den trainierten Gewichten ein gleitender Durchschnitt geführt, der für die finale Evaluation und das Sampling verwendet wird. Zusätzlich wird Gradient Clipping mit einer maximalen Norm von 1,0 angewendet, um Gradientenexplosionen zu verhindern.

Der Datensatz wird in 90% Trainings- und 10% Validierungsdaten aufgeteilt. Die Aufteilung erfolgt mit einem festen Random Seed, um die Reproduzierbarkeit sicherzustellen. Während des Trainings wird der Validierungsverlust nach jeder Epoche berechnet.

7.7 Ablationsstudien

Um den Einfluss einzelner Komponenten auf die Generierungsqualität zu untersuchen werden ausgehend von der Baseline-Konfiguration gezielt einzelne Variablen verändert. Pro Experiment wird jeweils nur eine Komponente angepasst, um deren isolierten Einfluss bewerten zu können. Die Ablationsstudien werden auf einer Auflösung von 32^3 Voxel durchgeführt.

Ohne Self-Attention Die Self-Attention-Schichten auf den niedrigen Auflösungsstufen werden entfernt. Dadurch lässt sich untersuchen, welchen Beitrag die Modellierung globaler Abhängigkeiten zur Bildqualität leistet und ob die lokalen Informationen der Faltungsschichten allein ausreichen.

Linearer Schedule Der Cosine Schedule wird durch einen linearen Schedule ersetzt, bei dem die Varianz gleichmäßig von $\beta_1 = 0,0001$ bis $\beta_T = 0,02$ ansteigt.

Noise-Prediction (ϵ -Prediction) Anstelle der v-Prediction wird das Modell darauf trainiert, das hinzugefügte Rauschen ϵ vorherzusagen.

Datenaugmentierung Die Trainingsdaten werden mit einer Wahrscheinlichkeit von 50% horizontal gespiegelt, was die approximative Links-Rechts-Symmetrie des Gehirns ausnutzt. Damit wird die effektive Datensatzgröße verdoppelt, ohne anatomisch unplausible Volumina zu erzeugen.

Erhöhte Auflösung (48^3). Zusätzlich wird die Baseline-Konfiguration auf einer Auflösung von 48^3 Voxeln trainiert, um den Einfluss der räumlichen Auflösung auf die Bildqualität zu untersuchen.

7.8 Aufgezeichnete Werte während des Trainings

Während des Trainings werden für jede Epoche verschiedene Metriken protokolliert, um den Trainingsverlauf nachvollziehen zu können. Dazu gehören der Trainings- und Validierungsverlust, die aktuelle Lernrate, die Epochendauer sowie der GPU-Speicherverbrauch. Zusätzlich werden die Gradientennorm und die Anzahl der durch Gradient Clipping begrenzten Updates erfasst, um die Stabilität des Trainings beurteilen zu können.

An festgelegten Epochen, bei 1 und 10 sowie anschließend alle 25 Epochen bis Epoche 300, werden Samples generiert, um die visuelle Qualität der erzeugten Volumina im Trainingsverlauf zu überprüfen. An denselben Epochen wird jeweils ein Checkpoint gespeichert; zusätzlich wird das Modell mit dem besten Validierungsverlust separat gesichert. Sämtliche Metriken werden nach jeder Epoche in der JSON-Datei aktualisiert, so dass der Trainingsverlauf auch nachträglich vollständig analysiert werden kann.

TODO: Später auf Anhang referenzieren welche Werte tatsächlich aufgenommen worden sind

7.9 Conditional-Modelle

Neben dem unconditional trainierten Baseline-Modell werden zwei conditional Varianten trainiert, die der gleichen 3D-UNet-Architektur folgen, jedoch um eine Konditionierungs-Schnittstelle erweitert sind. Beide Modelle nutzen Classifier-Free Guidance, bei dem die Konditionierung während des Trainings mit einer Wahrscheinlichkeit von 15% durch einen Null-Token ersetzt wird. Beim Sampling wird der Einfluss der Konditionierung mit einer Guidance Scale von $w = 3,0$ verstärkt.

Die übrigen Hyperparameter aus Abschnitt 7.6 werden übernommen. Lediglich das Gradient Clipping wird auf eine maximale Norm von 0,5 herabgesetzt, um das Training trotz des zusätzlichen Konditionierungssignals stabil zu halten.

Maske-zu-MRT-Modell Das Modell erzeugt ein synthetisches T1N-Volumen, das auf eine vorgegebene Tumor-Segmentierungsmaske konditioniert ist. Die Maske wird vom diskreten Wertebereich $\{0, \dots, 3\}$ auf das Intervall $[-1, 1]$ skaliert, damit sie sich auf demselben Wertebereich wie das verrauschte Eingangsvolumen bewegt und anschließend kanalweise an dieses angehängt. Das Modell sieht somit zwei Eingangskanäle: das verrauschte MRT und die zugehörige Segmentierung. Trainiert wird auf den paarweise vorliegenden $(T1N, Seg)$ -Volumina des BraTS-2023-Datensatzes.

Klassen-zu-Maske-Modell Das Modell erzeugt eine synthetische Tumor-Segmentierungsmaske aus einer vorgegebenen Größenklasse. Die Klassen sind in drei Stufen unterteilt: klein (≤ 2 cm), mittel (2–5 cm) und groß (> 5 cm) [85]. Die Grenzwerte sind in Anlehnung an die TNM-Klassifikation des Mammakarzinoms gewählt, da für Hirntumore keine etablierte größenbasierte TNM-Klassifikation existiert. Sie werden vorab anhand der Tumor-Ausdehnung in den realen Segmentierungen vergeben. Die Klassen-Information wird über eine Embedding-Schicht in einen Vektor abgebildet und additiv zur sinusoidalen Zeit-Einbettung hinzugefügt. Das Modell liefert kontinuierliche Werte in $[-1, 1]$, die für die weitere Verwendung in die vier diskreten BraTS-Labels zurückquantisiert werden.

7.10 Generierungs-Pipeline

Um vollständig synthetische Gehirn-Volumina mit realistisch positioniertem Tumor zu erzeugen, werden die beiden in Abschnitt 7.9 beschriebenen Modelle in einer dreistufigen Pipeline gekoppelt. Da das Klassen-zu-Maske-Modell auf zentrierten Tumormasken trainiert wurde, sitzen seine Samples in der Mitte des Volumens. In realen MRT-Aufnahmen liegen Tumore jedoch an unterschiedlichen anatomischen Positionen, weshalb zwischen den beiden Modellen eine räumliche Verschiebung eingefügt wird.

1. Tumormaske Das Klassen-zu-Maske-Modell erzeugt aus einer vorgegebenen Größenklasse eine am Volumenmittelpunkt zentrierte Tumormaske im Wertebereich $[-1, 1]$. Diese wird anschließend per Nearest-Neighbor-Zuordnung auf die vier diskreten BraTS-Labels zurückquantisiert.

2. Räumliche Platzierung Aus dem realen BraTS-Datensatz wird ein Segmentierungsvolumen zufällig ausgewählt und dessen Tumormittelpunkt berechnet. Die zentrierte synthetische Maske wird so verschoben, dass ihr Mittelpunkt mit diesem realen Mittelpunkt zusammenfällt. Würde durch diese Verschiebung ein Teil der Tumor-Bounding-Box aus dem Volumen herausragen, wird der Verschiebungsvektor entlang der jeweiligen Achse beschnitten. Die Verschiebung selbst erfolgt per Nearest-Neighbor-Interpolation, sodass die diskreten Labelwerte erhalten bleiben. Damit folgt die räumliche Verteilung der generierten Tumore der Verteilung realer Tumore, ohne dass anatomisch unmögliche Positionen entstehen.

3. Gehirn-Generierung Die platzierte Tumormaske wird auf das Intervall $[-1, 1]$ skaliert und dient als Konditionierung für das Maske-zu-MRT-Modell, welches das umliegende T1N-Gehirn-Volumen generiert. Pro Pipeline-Durchlauf werden zwei NIfTI-Dateien gespeichert: die platzierte Tumormaske und das synthetisierte Gehirn-Volumen.

Für jede der drei Stufen wird ein separater Seed verwendet, sodass Tumorform, räumliche Platzierung und Gehirntextur unabhängig voneinander kontrolliert werden können.

TODO: Abbildung!!

7.11 Sample-Generierung

Nach Abschluss des Trainings werden die Modelle zur Generierung synthetischer Volumina eingesetzt. Pro Modell und Sampler werden 1024 Samples erzeugt. Für das Sampling werden die EMA-Gewichte aus Abschnitt 7.6 verwendet.

Sampler Als Sampling-Verfahren stehen vier Schemata aus der *diffusers*-Bibliothek zur Verfügung: DDPM [44], DDIM [46], DPM-Solver++ [86] und UniPC [48]. DDPM durchläuft den vollständigen Trainings-Schedule mit 250 Schritten, während die schnelleren Solver mit deutlich weniger Schritten auskommen. DDIM mit 50, DPM-Solver++ mit 25 und UniPC mit 10. Pro generiertem Sample wird ein eigener Seed verwendet, sodass die Volumina untereinander unabhängig und der Vorgang reproduzierbar bleibt.

Unconditional Bei den unconditional trainierten Modellen startet jeder Sample-Vorgang aus einem unabhängig gezogenen Gaußschen Rauschtensor. Gestartet werden sowohl die fünf 32^3 -Ablationsvarianten als auch die 48^3 -Baseline.

Conditional Für das Maske-zu-MRT-Modell wird pro Sample eine Tumormaske zufällig aus den vorverarbeiteten BraTS-Segmentierungen gezogen. Das Klassen-zu-Maske-Modell durchläuft hingegen die drei Größenklassen nacheinander, sodass alle gleich oft vertreten sind.

Pipeline Das Vorgehen für die Pipeline-Generierung ist in Abschnitt 7.10 beschrieben. Sie kann wahlweise mit zyklisch durchlaufenen oder mit einer fest vorgegebenen Größenklasse betrieben werden.

Aufgezeichnete Werte während des Samplings Pro Sample werden der verwendete Seed, die Generierungsdauer sowie die Voxel-Statistiken (Minimum, Maximum, Mittelwert, Standardabweichung) festgehalten. Bei den conditional Modellen wird zusätzlich die verwendete Konditionierung erfasst: die Datei der Eingangsmaske beim Maske-zu-MRT-Modell und die Größenklasse beim Klassen-zu-Maske-Modell. Für die Pipeline kommen der Referenz-Tumormittelpunkt aus dem realen Datensatz sowie der angewandte Verschiebungsvektor hinzu. Pro Generierungslauf werden außerdem die Sampler-Konfiguration und der Peak-GPU-Speicherverbrauch in einer JSON-Datei abgelegt.

TODO: Werte nochmal durchlesen. Abändern in den Anhang

7.12 Testdurchlauf

Nach der Sample-Generierung werden die in Unterabschnitt 6.4.2 vorgestellten Metriken auf die erzeugten Datensätze angewendet. Die Auswertung läuft über ein eigenständi-

ges Jupyter-Notebook, das nach der Sample-Generierung gestartet wird. Es wählt je nach gewähltem Modus die passende Paarungsstrategie und legt die Ergebnisse sowohl pro Sample als auch aggregiert als CSV-Datei ab.

Auswertungsmodi Das Notebook unterscheidet vier Modi. *Unconditional* bewertet die unconditional erzeugten MRT-Volumina gegen den realen BraTS-Datensatz. *Maske zu MRT* vergleicht die Ausgaben des Maske-zu-MRT-Modells mit den echten T1N-Volumina derselben Probanden. *Klasse zu Maske* setzt die generierten Tumormasken gegen reale Masken der gleichen Größenklasse. *Pipeline* schließlich bewertet die voll-synthetischen Gehirn-Volumina aus der Generierungs-Pipeline gegen den realen T1N-Datensatz.

Paarung Da die Full-Reference-Metriken SSIM, PSNR und LPIPS ein Referenzbild voraussetzen, muss jedem generierten Volumen ein reales Gegenstück zugeordnet werden. Bei *Unconditional* und *Pipeline* ist keine inhärente Zuordnung gegeben, daher wird mit festem Seed zufällig gepaart. Im Modus *Maske zu MRT* ermöglicht die in der `stats.json` mitgeschriebene Samples eine exakte Zuordnung zu den realen T1N-Volumina derselben Subjekte. *Klasse zu Maske* paart ebenfalls zufällig, jedoch nur innerhalb der gleichen Größenklasse, damit die Klassenverteilung erhalten bleibt.

Berechnung der Metriken SSIM und PSNR werden direkt auf den 3D-Volumina mittels der `scikit-image`-Bibliothek berechnet. Der Wertebereich ist auf $\Delta = 2,0$ gesetzt, da die Volumina im Intervall $[-1, 1]$ liegen. LPIPS arbeitet auf zweidimensionalen Bildern (vortrainiertes AlexNet [77]) und wird daher schichtweise auf die axialen Schnitte angewendet und über diese gemittelt. FID nutzt ein Inception-V3-Netzwerk ohne Klassifikationskopf. Die axialen Schnitte werden auf 299×299 Pixel skaliert, im RGB-Raum repliziert und mit ImageNet-Statistiken normalisiert, bevor die 2048-dimensionalen Merkmalsvektoren entnommen werden. Der Dice-Koeffizient kommt ausschließlich im Modus *Klasse zu Maske* zum Einsatz: die generierten Werte werden auf die diskreten BraTS-Labels zurückquantisiert, anschließend Dice pro Vordergrundklasse berechnet und über die drei Tumor-Subregionen gemittelt.

Ergebnisausgabe Pro Auswertungslauf werden zwei CSV-Dateien geschrieben. Die Aggregat-Datei enthält eine Zeile mit Mittelwert und Standardabweichung sämtlicher Metriken. Die Detail-Datei führt die Werte pro Sample auf.

8 Ergebnis

Dieses Kapitel stellt die Ergebnisse der durchgeführten Experimente vor. Zunächst werden die während des Trainings aufgezeichneten Verlaufsmetriken berichtet, anschließend die quantitativen Auswertungen der einzelnen Modelle. Den Abschluss bildet eine qualitative Gegenüberstellung ausgewählter generierter Volumina. Eine Einordnung und Diskussion der Ergebnisse erfolgt in Kapitel 9.

8.1 Trainingsverlauf

Insgesamt wurden acht Modelle mit 300 Epochen trainiert: fünf 32^3 -Ablationsvarianten, die 48^3 -Baseline sowie die beiden conditional Modelle für die in Abschnitt 7.10 beschriebene Generierungs-Pipeline. Tabelle ?? fasst die zentralen Verlaufsmetriken zusammen.

Modell	Dauer (h)	Batch	Train-Loss	Val-Loss	Min. Val-Loss	Beste Epoche
<i>Unconditional 32^3</i>						
Baseline	56,05	10	0,00051	0,1092	0,0573	45
Data Augmentation	55,80	10	0,00164	0,1039	0,0612	102
Linearer Schedule	57,53	10	0,00052	0,0995	0,0568	39
Ohne Self-Attention	56,61	10	0,00071	0,1102	0,0579	45
Noise-Prediction	58,07	10	0,00118	0,0549	0,0366	76
<i>Unconditional 48^3</i>						
Baseline	187,41	3	0,00821	0,0611	0,0445	276
<i>Conditional 32^3</i>						
Maske $\rightarrow$ MRT	62,56	10	0,00100	0,1095	0,0631	54
Klasse $\rightarrow$ Maske	63,72	10	0,00006	0,0211	0,0168	158

Tabelle 8.1: Zusammenfassung des Trainingsverlaufs aller Modelle über 300 Epochen. *Beste Epoche* bezeichnet die Epoche mit dem niedrigsten Validierungs-Loss.

Vereinzelte Loss-Spikes traten bei fünf der acht Modelle auf, jeweils ohne nachfolgende Destabilisierung. Das Klasse-zu-Maske-Modell wies mit dreizehn Spikes die meisten Ereignisse auf, das 48^3 -Baseline-Modell zusätzlich fünf isolierte NaN-/Inf-Werte. Detaillierte epochenweise Verläufe sind im Anhang aufgeführt.

Visualisierung Abb. 8.1 stellt die Trainings- und Validierungsverläufe aller Modelle über 300 Epochen gegenüber. Abb. 8.2 zeigt die benötigte GPU-Zeit.

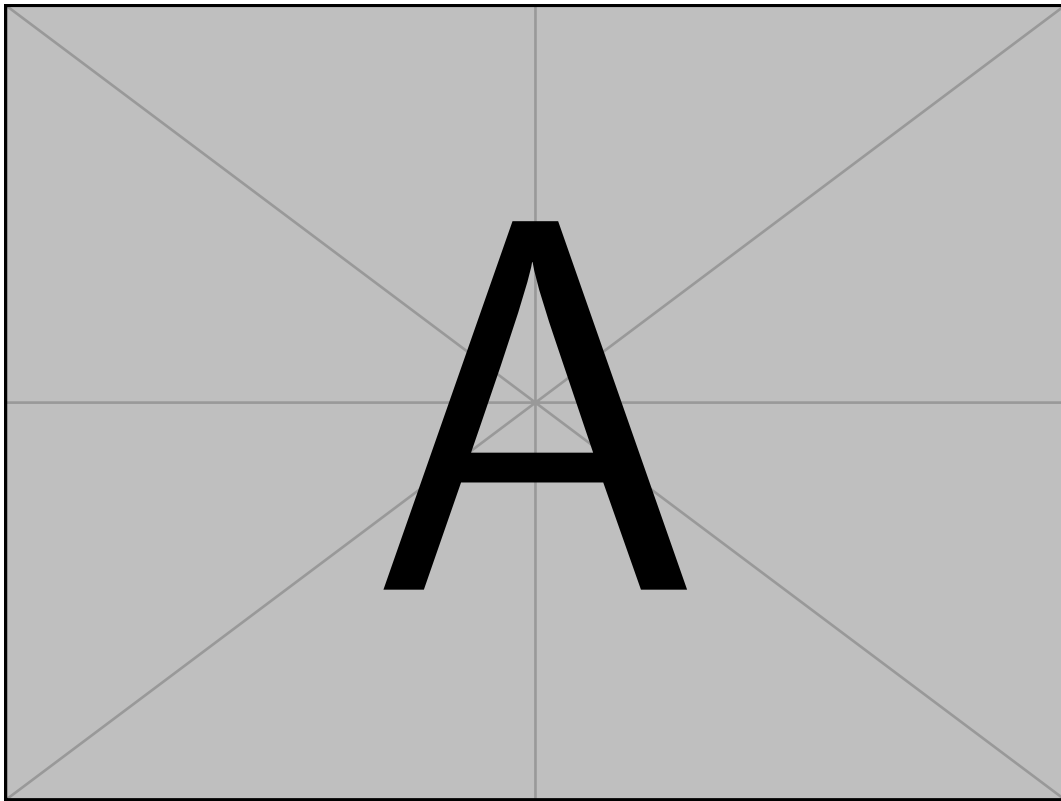


Abb. 8.1: Trainings- (durchgezogen) und Validierungs-Loss (gestrichelt) über 300 Epochen für alle acht Modelle.

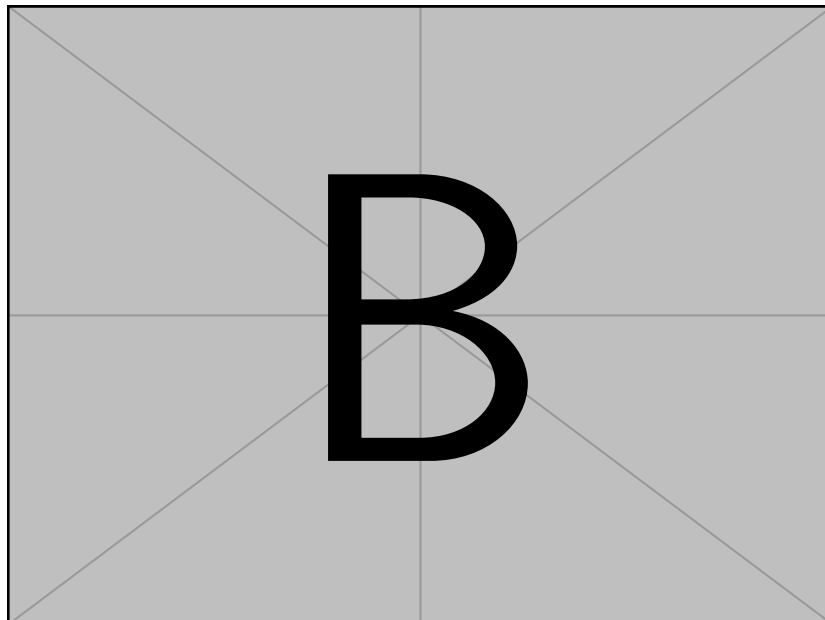


Abb. 8.2: Trainingsdauer der acht Modelle in GPU-Stunden auf einer NVIDIA GeForce GTX 1080 Ti.

8.2 Quantitative Auswertung der unconditional Modelle

Pro Modell und Sampler wurden 1024 Samples generiert und gegen den realen BraTS-2023-Datensatz mit den in Abschnitt 7.6 beschriebenen Metriken ausgewertet. Die FID wurde über alle 1251 realen Volumina als Referenzmenge berechnet; SSIM, PSNR und LPIPS wurden paarweise gegen ein zufällig zugeordnetes reales Volumen bestimmt. Tabelle ?? fasst die Ergebnisse zusammen. Der jeweils beste Wert pro Modell und Metrik ist fett hervorgehoben.

Modell	Sampler	FID ↓	SSIM ↑	PSNR ↑	LPIPS ↓
Baseline	DDPM	24,77	0,504	17,98	0,035
	DDIM	44,75	0,501	17,81	0,037
	DPM-Solver++	40,95	0,503	17,85	0,037
	UniPC	43,93	0,503	17,85	0,037
Data Augmentation	DDPM	37,91	0,463	16,41	0,036
	DDIM	40,19	0,456	16,43	0,038
	DPM-Solver++	29,60	0,465	16,57	0,037
	UniPC	27,90	0,469	16,65	0,037
Linearer Schedule	DDPM	73,95	0,238	12,85	0,089
	DDIM	130,62	0,144	10,91	0,144
	DPM-Solver++	129,23	0,147	10,98	0,140
	UniPC	133,76	0,145	10,97	0,140
Ohne Self-Attention	DDPM	30,77	0,516	18,08	0,035
	DDIM	48,57	0,510	17,86	0,038
	DPM-Solver++	45,35	0,511	17,89	0,037
	UniPC	50,70	0,509	17,89	0,038
Noise-Prediction	DDPM	217,16	-0,029	3,82	0,507
	DDIM	260,40	-0,059	7,39	0,363
	DPM-Solver++	489,92	-0,023	3,86	0,520
	UniPC	497,78	-0,022	3,87	0,522

Tabelle 8.2: Quantitative Auswertung der fünf 32^3 -Modelle über 1024 generierte Samples je Sampler. Niedrigere Werte sind besser bei FID und LPIPS, höhere bei SSIM und PSNR. Beste Werte je Modell sind fett markiert.

Über alle 32^3 -Konfigurationen weist die Baseline mit DDPM den niedrigsten Gesamt-FID (24,77) auf. Die niedrigsten FID-Werte je Modell werden bei vier von fünf Varianten mit dem DDPM-Sampler erreicht. Einzige Ausnahme ist das Data-Augmentation-Modell, bei dem UniPC mit 27,90 am besten abschneidet. Die Noise-Prediction-Variante liefert in allen vier Samplern negative SSIM-Werte und PSNR-Werte unter 8 dB.

Visualisierung der Metrikverteilung Abb. 8.3 stellt die Verteilung der pro Sample berechneten Metriken SSIM, PSNR und LPIPS als Boxplots dar.

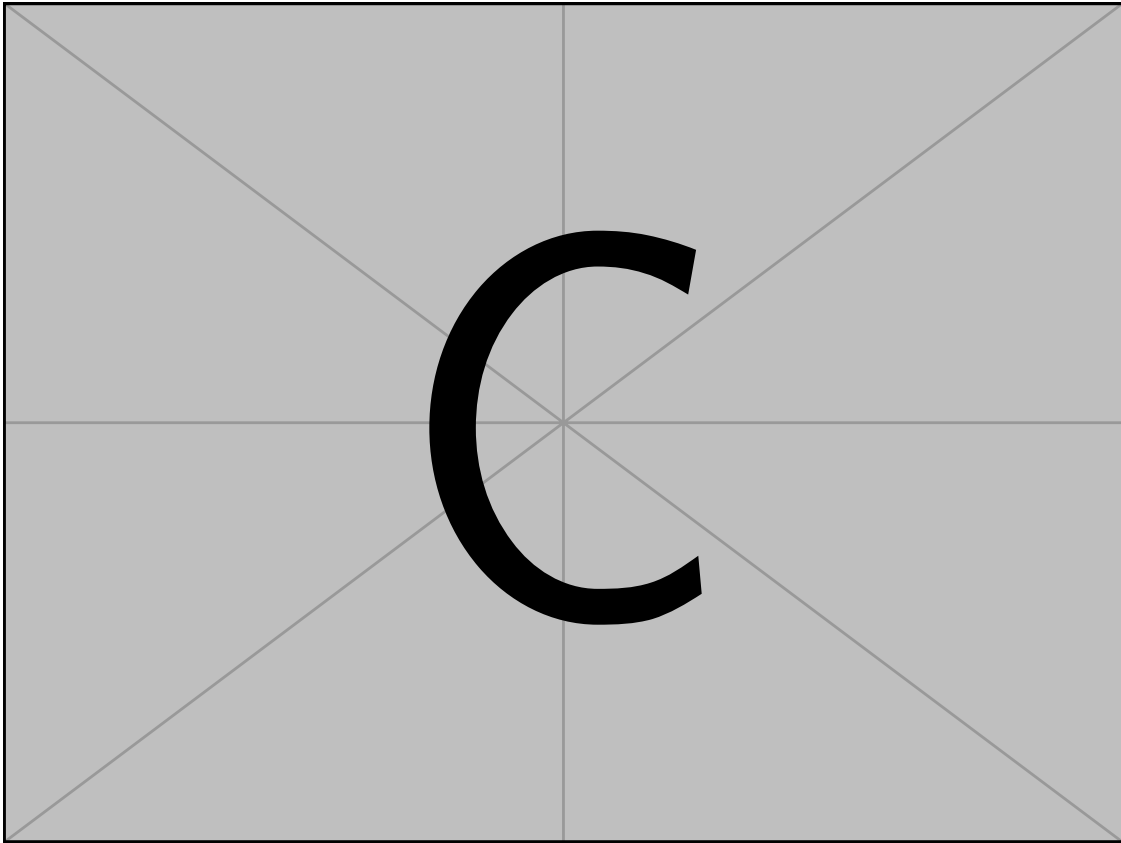


Abb. 8.3: Verteilung der paarweisen Metriken über 1024 Samples für die fünf 32^3 -Modelle, dargestellt jeweils für den FID-besten Sampler je Modell.

8.3 Erhöhte Auflösung (48^3)

Die Baseline wurde zusätzlich in einer Auflösung von 48^3 Voxeln trainiert und mit denselben vier Samplern ausgewertet. Tabelle 8.3 zeigt die Ergebnisse.

Tabelle 8.3: Quantitative Auswertung der 48^3 -Baseline über 1024 generierte Samples je Sampler. Beste Werte je Spalte sind fett markiert.

Sampler	FID ↓	SSIM ↑	PSNR ↑	LPIPS ↓
DDPM	57,11	0,525	16,41	0,117
DDIM	65,42	0,465	15,69	0,147
DPM-Solver++	63,44	0,484	15,94	0,138
UniPC	65,97	0,483	16,02	0,142

Wie bei den 32^3 -Modellen erreicht der DDPM-Sampler die niedrigste FID. Der SSIM der 48^3 -Variante mit DDPM (0,525) liegt oberhalb des entsprechenden Werts der 32^3 -Baseline (0,504); PSNR und LPIPS fallen in der höheren Auflösung niedriger aus.

8.4 Generierungs-Pipeline

Die in Abschnitt 7.10 beschriebene Pipeline aus Klasse-zu-Maske- und Maske-zu-MRT-Modell wurde ebenfalls für 1024 Durchläufe je Sampler ausgewertet. Die so erzeugten voll-synthetischen Hirnvolumina wurden zufällig mit realen T1N-Volumina gepaart und mit denselben Metriken bewertet. Tabelle 8.4 fasst die Ergebnisse zusammen.

Tabelle 8.4: Quantitative Auswertung der Generierungs-Pipeline (Klasse $\rightarrow$ Maske $\rightarrow$ MRT) in 32^3 über 1024 generierte Samples je Sampler. Beste Werte je Spalte sind fett markiert.

Sampler	FID $\downarrow$	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$
DDPM	28,82	0,520	18,04	0,034
DDIM	36,88	0,508	17,79	0,038
DPM-Solver++	28,45	0,512	17,84	0,037
UniPC	25,46	0,511	17,85	0,036

Die niedrigste FID der Pipeline wird mit UniPC bei 25,46 erreicht. Die paarweisen Metriken SSIM, PSNR und LPIPS der Pipeline-Samples liegen in derselben Größenordnung wie die der unconditional Baseline in 32^3 .

8.5 Qualitative Ergebnisse

Im Folgenden werden ausgewählte Sample-Volumina dargestellt. Jede Abbildung zeigt einen axialen Mittelschnitt pro Sample. Die Auswahl der Samples erfolgte zufällig mit festem Seed, ohne nachträgliche Selektion.

Unconditional Modelle (32^3) Abb. 8.4 stellt je acht Samples pro Ablationsvariante gegenüber, generiert mit dem jeweils FID-besten Sampler.

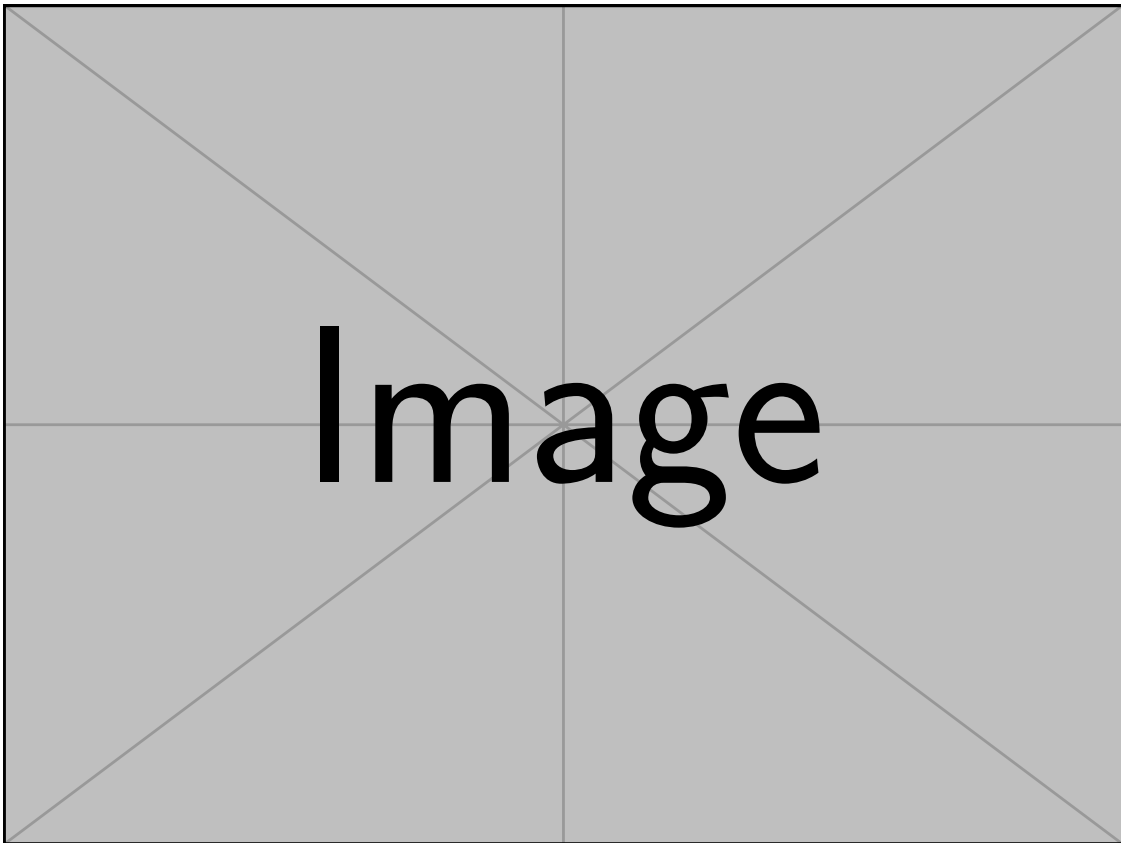


Abb. 8.4: Je acht zufällig ausgewählte Samples der fünf 32^3 -Modelle (axialer Mittelschnitt). Zeilen von oben nach unten: Baseline, Data Augmentation, Linearer Schedule, Ohne Self-Attention, Noise-Prediction.

Vergleich der Auflösungen Abb. 8.5 stellt die Baseline in beiden Auflösungen direkt gegenüber.

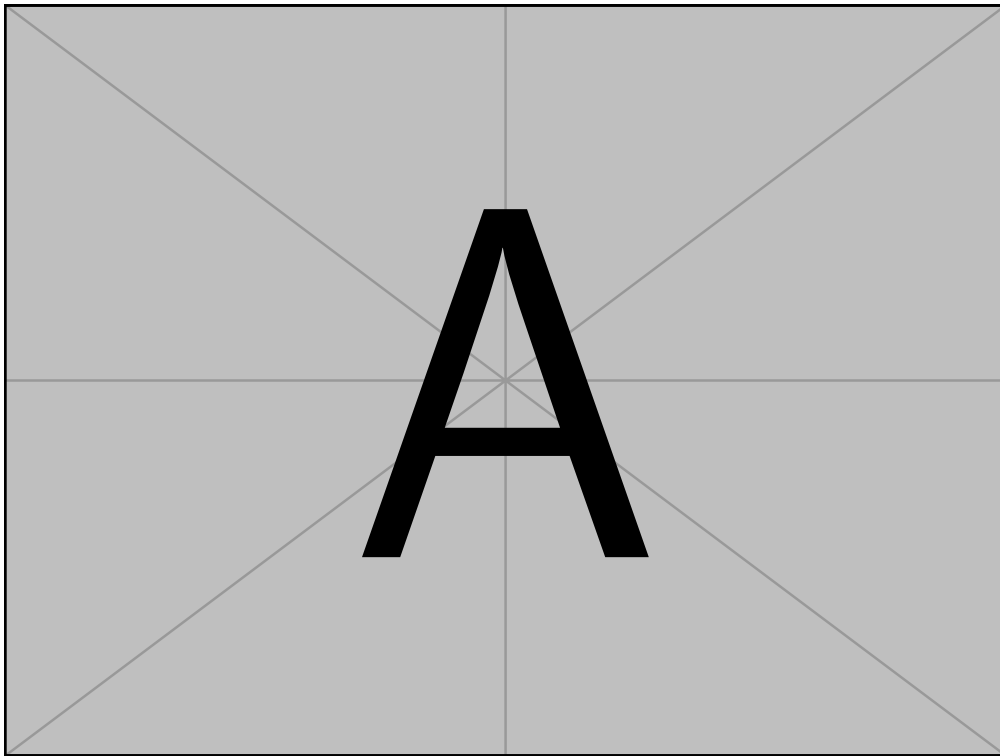


Abb. 8.5: Vergleich der Baseline in 32^3 (obere Reihe) und 48^3 (untere Reihe), je sechs zufällig ausgewählte Samples mit DDPM-Sampler.

Vergleich der Sampler Abb. 8.6 zeigt dasselbe Startrauschen, also den gleichen Seed, durch die vier Sampler verarbeitet.

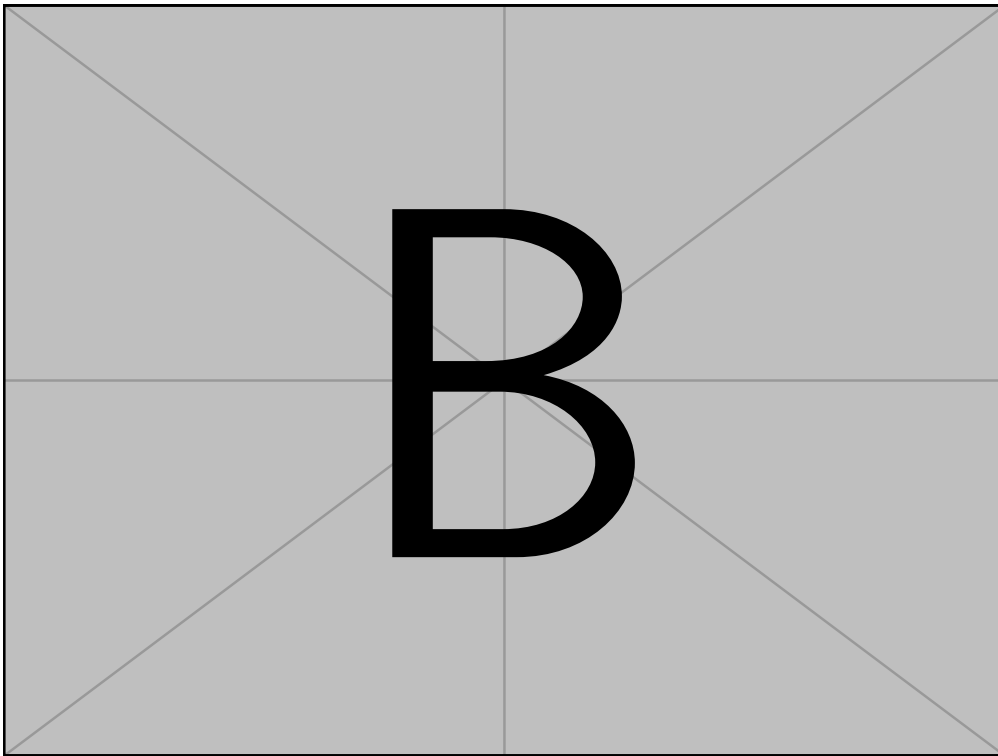


Abb. 8.6: Identisches Startrauschen für die Baseline 32^3 , dekodiert durch DDPM (250 Schritte), DDIM (50), DPM-Solver++ (25) und UniPC (10).

Generierungs-Pipeline Abb. 8.7 zeigt die End-zu-End-Ausgabe der Pipeline, jeweils mit der zugehörigen generierten Tumormaske als Überlagerung.

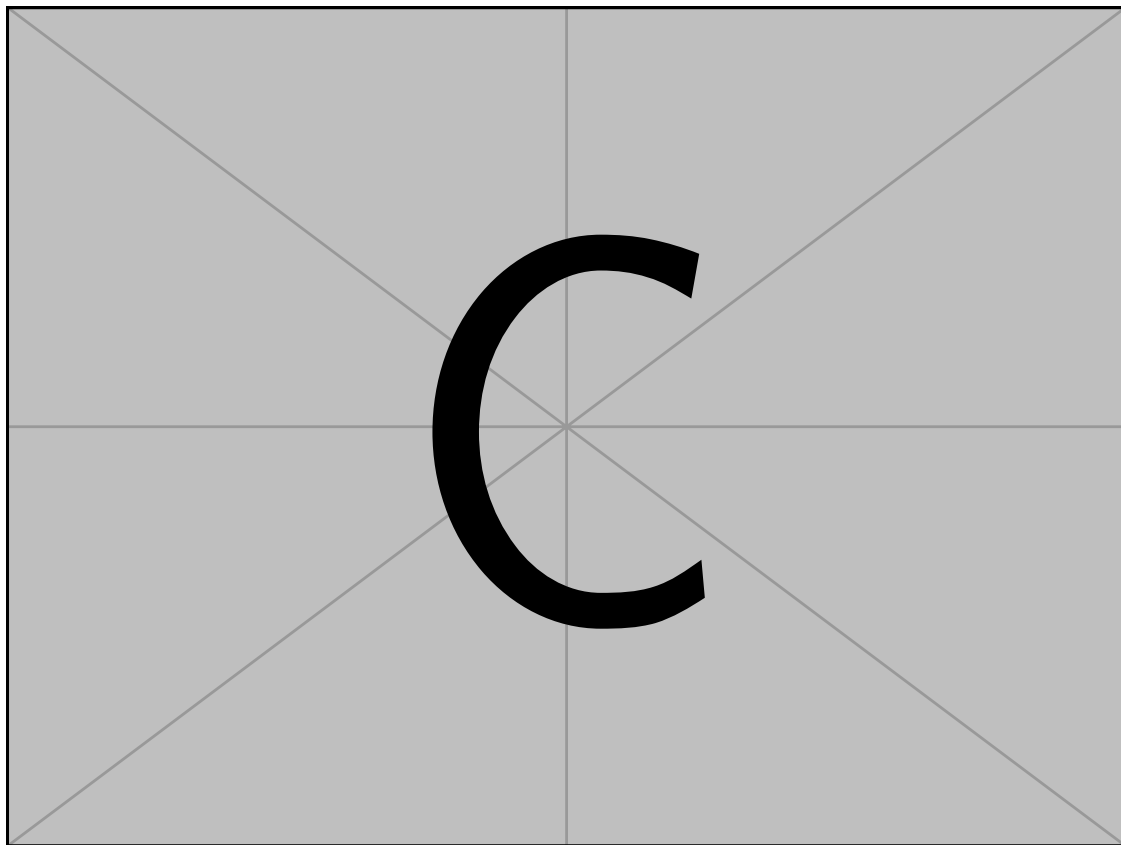


Abb. 8.7: Sechs Pipeline-Ausgaben mit UniPC: generierte Tumormaske (links), zugehöriges generiertes T1N-Volumen (Mitte) und Überlagerung beider (rechts).

Denoising-Trajektorie Abb. 8.8 zeigt den iterativen Übergang von reinem Rauschen zum finalen Volumen.

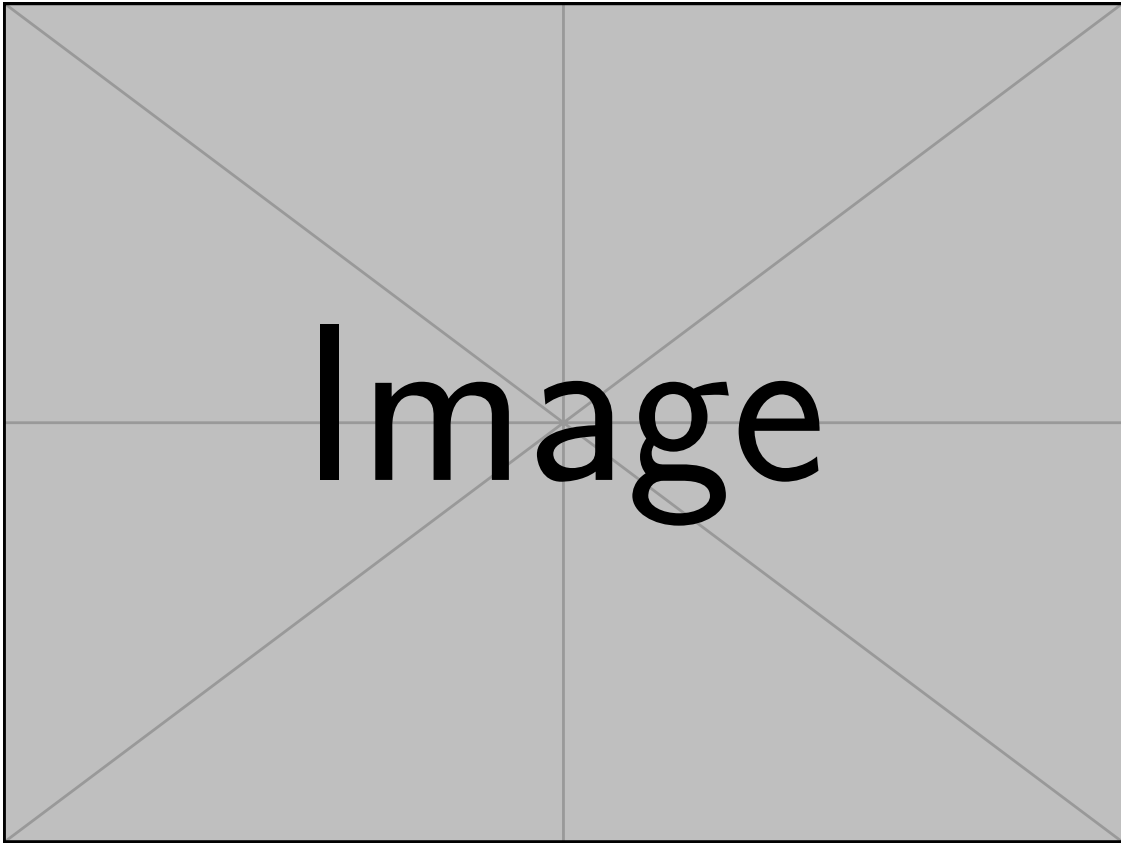


Abb. 8.8: Denoising-Trajektorie eines Samples der Baseline 32^3 mit DDIM, dargestellt an acht äquidistanten Zwischenschritten von $t = T$ (links) bis $t = 0$ (rechts).

8.6 Sample-Generierung

Für jede Modell-Sampler-Kombination wurden 1024 Volumina generiert. Die in Abschnitt 7.11 eingeführten Sampler unterscheiden sich in der Anzahl der Denoising-Schritte (DDPM: 250, DDIM: 50, DPM-Solver++: 25, UniPC: 10) und damit in der Generierungsdauer. Tabelle 8.5 fasst die gemessene Laufzeit und den Peak-VRAM-Verbrauch der unconditional Baseline für beide Auflösungen zusammen.

Auflösung	Sampler	Schritte	Ø Zeit/Sample (s)	Gesamt (min)	Peak VRAM (GB)
32 ³	DDPM	250	47,81	816,0	3,86
	DDIM	50	9,28	158,3	3,89
	DPM-Solver++	25	4,60	78,5	3,86
	UniPC	10	1,68	28,7	4,28
48 ³	DDPM	250	151,45	2582,3	6,52
	DDIM	50	29,98	511,6	4,26
	DPM-Solver++	25	14,92	254,7	4,66
	UniPC	10	5,70	97,3	4,34

Tabelle 8.5: Generierungsleistung der vier Sampler für die unconditional Baseline auf einer NVIDIA GeForce GTX 1080 Ti, gemittelt über 1024 generierte Volumina je Konfiguration. Peak-VRAM laut `nvidia-smi` (Gesamtbelegung der GPU). Vollständige Werte für die übrigen Ablationsvarianten siehe Anhang.

Von DDPM (250 Schritte) zu UniPC (10 Schritte) sinkt die mittlere Generierungsdauer pro Sample bei 32³ um den Faktor 28,5 und bei 48³ um den Faktor 26,6. Bei einer Erhöhung der Voxel-Kantenlänge von 32 auf 48 verlängert sich die Generierungsdauer um den Faktor 3,2 (DDPM) bis 3,4 (UniPC). Der Peak-VRAM-Verbrauch liegt bei 32³ im Bereich von 3,86 bis 4,28 GB und bei 48³ zwischen 4,26 und 6,52 GB.

Sampler	Schritte	Klasse → Maske (s)	Maske → MRT (s)	Gesamt (s)	Peak VRAM (GB)
DDPM	250	47,72	47,77	95,49	6,87
DDIM	50	9,02	9,01	18,03	6,89
DPM-Solver++	25	4,40	4,45	8,85	6,89
UniPC	10	1,81	1,82	3,63	6,90

Tabelle 8.6: Generierungsleistung der vier Sampler für die Generierungs-Pipeline (Klasse → Maske → MRT) in 32³, gemittelt über 1024 Pipeline-Durchläufe. Die Gesamtdauer setzt sich aus den Laufzeiten der beiden conditional Modelle zusammen.

In Tabelle 8.6 sind die Laufzeiten der Generierungs-Pipeline aus Klasse-zu-Maske- und Maske-zu-MRT-Modell dargestellt. Die mittlere Gesamtdauer pro Pipeline-Durchlauf beträgt 95,49 s (DDPM), 18,03 s (DDIM), 8,85 s (DPM-Solver++) und 3,63 s (UniPC). Der Peak-VRAM-Verbrauch der Pipeline liegt für alle vier Sampler ungefähr bei 6,9 GB.

9 Auswertung

10 Schlussfolgerung

10.1 Fazit

10.2 Ausblick

Literatur

- [1] Richard Bitar u. a. „MR Pulse Sequences: What Every Radiologist Wants to Know but Is Afraid to Ask“. en. In: *RadioGraphics* 26.2 (März 2006), S. 513–537. ISSN: 0271-5333, 1527-1323. DOI: 10.1148/rg.262055063. URL: <http://pubs.rsna.org/doi/10.1148/rg.262055063> (besucht am 16.05.2026).
- [2] Anahita Fathi Kazerooni u. a. „The Brain Tumor Segmentation (BraTS) Challenge 2023: Focus on Pediatrics (CBTN-CONNECT-DIPGR-ASNR-MICCAI BraTS-PEDs)“. en. In: ().
- [3] Thorsten M. Buzug. *Computed Tomography. From Photon Statistics to Modern Cone-Beam CT*. 1. Aufl. Berlin, Heidelberg: Springer, 2008, S. XIV, 522. ISBN: 978-3-540-39408-2. DOI: 10.1007/978-3-540-39408-2. URL: <https://doi.org/10.1007/978-3-540-39408-2>.
- [4] Sridaran Rajagopal u. a., Hrsg. *Artificial Intelligence Based Smart and Secured Applications: 4th International Conference, ASCIS 2025, Gujarat, India, September 11–13, 2025, Revised Selected Papers, Part III*. en. Bd. 2821. Communications in Computer and Information Science. Cham: Springer Nature Switzerland, 2026. ISBN: 978-3-032-17839-8 978-3-032-17840-4. DOI: 10.1007/978-3-032-17840-4. URL: <https://link.springer.com/10.1007/978-3-032-17840-4> (besucht am 18.02.2026).
- [5] Ahmed Elhadad, Mona Jamjoom und Hussein Abulkasim. „Reduction of NIFTI files storage and compression to facilitate telemedicine services based on quantization hiding of downsampling approach“. en. In: *Scientific Reports* 14.1 (März 2024), S. 5168. ISSN: 2045-2322. DOI: 10.1038/s41598-024-54820-4. URL: <https://www.nature.com/articles/s41598-024-54820-4> (besucht am 18.02.2026).
- [6] Department of Computer Applications, Kalasalingam Academy of Research and Education (Deemed to be University), Krishnankoil, Tamil Nadu, India. u. a. „An Medical Image File Formats and Digital Image Conversion“. en. In: *International Journal of Engineering and Advanced Technology* 9.1s4 (Dez. 2019), S. 74–78. ISSN: 22498958. DOI: 10.35940/ijeat.A1093.1291S419. URL: <https://www.ijeat.org/portfolio-item/A10931291S419/> (besucht am 18.02.2026).
- [7] Harald Nahrstedt. *Algorithmen für Ingenieure*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2018. ISBN: 978-3-658-19298-3 978-3-658-19299-0. DOI: 10.1007/978-3-658-19299-0. URL: <http://link.springer.com/10.1007/978-3-658-19299-0> (besucht am 10.10.2025).
- [8] Andreas Kohne u. a. *Chatbots: Aufbau und Anwendungsmöglichkeiten von autonomen Sprachassistenten*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2020. ISBN: 978-3-658-28848-8 978-3-658-28849-5. DOI: 10.1007/978-3-658-28849-5.

- URL: <http://link.springer.com/10.1007/978-3-658-28849-5> (besucht am 06.10.2025).
- [9] Zhen LLeo" Liu. *Artificial Intelligence for Engineers: Basics and Implementations*. en. Cham: Springer Nature Switzerland, 2025. ISBN: 978-3-031-75952-9 978-3-031-75953-6. DOI: 10.1007/978-3-031-75953-6. URL: <https://link.springer.com/10.1007/978-3-031-75953-6> (besucht am 06.10.2025).
 - [10] Vasant Honavar. „Artificial Intelligence: An Overview“. en. In: (). (Besucht am 07.10.2025).
 - [11] Paolo Bory, Simone Natale und Christian Katzenbach. „Strong and weak AI narratives: an analytical framework“. en. In: *AI & SOCIETY* 40.4 (Apr. 2025), S. 2107–2117. ISSN: 0951-5666, 1435-5655. DOI: 10.1007/s00146-024-02087-8. URL: <https://link.springer.com/10.1007/s00146-024-02087-8> (besucht am 14.10.2025).
 - [12] Carsten Lanquillon und Sigurd Schacht. *Knowledge Science - Grundlagen: Methoden der Künstlichen Intelligenz für die Wissensextraktion aus Texten*. de. Wiesbaden: Springer Fachmedien Wiesbaden, 2023. ISBN: 978-3-658-41688-1 978-3-658-41689-8. DOI: 10.1007/978-3-658-41689-8. URL: <https://link.springer.com/10.1007/978-3-658-41689-8> (besucht am 06.10.2025).
 - [13] Robert Ciesla. *The Book of Chatbots: From ELIZA to ChatGPT*. en. Cham: Springer Nature Switzerland, 2024. ISBN: 978-3-031-51003-8 978-3-031-51004-5. DOI: 10.1007/978-3-031-51004-5. URL: <https://link.springer.com/10.1007/978-3-031-51004-5> (besucht am 14.10.2025).
 - [14] Tom M. Mitchell. *Machine learning*. en. Nachdr. McGraw-Hill series in Computer Science. New York: McGraw-Hill, 2013. ISBN: 978-0-07-042807-2 978-0-07-115467-3.
 - [15] Christopher M. Bishop. *Pattern recognition and machine learning*. en. Information science and statistics. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
 - [16] Stuart J. Russell und Peter Norvig. *Artificial intelligence: a modern approach*. en. Third edition, Global edition. Prentice Hall series in artificial intelligence. Boston Columbus Indianapolis: Pearson, 2016. ISBN: 978-0-13-604259-4 978-1-292-15397-1. (Besucht am 07.10.2025).
 - [17] Jeff Heaton. „Ian Goodfellow, Yoshua Bengio, and Aaron Courville: Deep learning: The MIT Press, 2016, 800 pp, ISBN: 0262035618“. en. In: *Genetic Programming and Evolvable Machines* 19.1-2 (Juni 2018), S. 305–307. ISSN: 1389-2576, 1573-7632. DOI: 10.1007/s10710-017-9314-z. URL: <http://link.springer.com/10.1007/s10710-017-9314-z> (besucht am 10.05.2026).
 - [18] Richard S Sutton und Andrew G Barto. „Reinforcement Learning: An Introduction“. en. In: ().
 - [19] Nils Urbach und Daniel Feulner, Hrsg. *Managing Artificial Intelligence: How Organizations Succeed with AI*. en. Future of Business and Finance. Cham: Springer Nature Switzerland, 2026. ISBN: 978-3-032-13307-6 978-3-032-13308-3. DOI: 10.1007/

- 978-3-032-13308-3. URL: <https://link.springer.com/10.1007/978-3-032-13308-3> (besucht am 24.03.2026).
- [20] F. Rosenblatt. *The Perceptron, a Perceiving and Recognizing Automaton: (Project Para)*. Report / Cornell Aeronautical Laboratory. Cornell Aeronautical Laboratory, 1957. URL: https://books.google.de/books?id=P_XGPgAACAAJ.
- [21] Hang Li. *Machine Learning Methods*. en. Singapore: Springer Nature Singapore, 2024. ISBN: 978-981-99-3916-9 978-981-99-3917-6. DOI: 10.1007/978-981-99-3917-6. URL: <https://link.springer.com/10.1007/978-981-99-3917-6> (besucht am 07.04.2026).
- [22] Marvin Minsky und Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: The MIT Press, 1969.
- [23] Andrew Y. Ng und Michael I. Jordan. „On Discriminative vs. Generative Classifiers: A Comparison of Logistic Regression and Naive Bayes“. In: *Advances in Neural Information Processing Systems 14 (NIPS 2001)*. Hrsg. von Thomas G. Dietterich, Suzanna Becker und Zoubin Ghahramani. Cambridge, MA, USA: MIT Press, 2002, S. 841–848.
- [24] Stefan Feuerriegel u. a. „Generative AI“. en. In: *Business & Information Systems Engineering* 66.1 (Feb. 2024), S. 111–126. ISSN: 2363-7005, 1867-0202. DOI: 10.1007/s12599-023-00834-7. URL: <https://link.springer.com/10.1007/s12599-023-00834-7> (besucht am 14.02.2026).
- [25] Chieh-Hsin Lai u. a. *The Principles of Diffusion Models*. en. arXiv:2510.21890 [cs]. Okt. 2025. DOI: 10.48550/arXiv.2510.21890. URL: <http://arxiv.org/abs/2510.21890> (besucht am 14.02.2026).
- [26] Lars Ruthotto und Eldad Haber. „An introduction to deep generative modeling“. en. In: *GAMM-Mitteilungen* 44.2 (Juni 2021), e202100008. ISSN: 0936-7195, 1522-2608. DOI: 10.1002/gamm.202100008. URL: <https://onlinelibrary.wiley.com/doi/10.1002/gamm.202100008> (besucht am 15.02.2026).
- [27] Tianhong Li, Dina Katabi und Kaiming He. *Return of Unconditional Generation: A Self-supervised Representation Generation Method*. en. arXiv:2312.03701 [cs]. Nov. 2024. DOI: 10.48550/arXiv.2312.03701. URL: <http://arxiv.org/abs/2312.03701> (besucht am 05.05.2026).
- [28] Minshuo Chen u. a. *An Overview of Diffusion Models: Applications, Guided Generation, Statistical Rates and Optimization*. en. arXiv:2404.07771 [cs]. Apr. 2024. DOI:

- 10.48550/arXiv.2404.07771. URL: <http://arxiv.org/abs/2404.07771> (besucht am 05.05.2026).
- [29] Prafulla Dhariwal und Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. en. arXiv:2105.05233 [cs]. Juni 2021. DOI: 10.48550/arXiv.2105.05233. URL: <http://arxiv.org/abs/2105.05233> (besucht am 07.05.2026).
- [30] Jonathan Ho und Tim Salimans. *Classifier-Free Diffusion Guidance*. en. arXiv:2207.12598 [cs]. Juli 2022. DOI: 10.48550/arXiv.2207.12598. URL: <http://arxiv.org/abs/2207.12598> (besucht am 07.05.2026).
- [31] Zizhao Zhang, Lin Yang und Yefeng Zheng. „Translating and Segmenting Multimodal Medical Volumes with Cycle- and Shape-Consistency Generative Adversarial Network“. en. In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. Salt Lake City, UT: IEEE, Juni 2018, S. 9242–9251. ISBN: 978-1-5386-6420-9. DOI: 10.1109/CVPR.2018.00963. URL: <https://ieeexplore.ieee.org/document/8579061/> (besucht am 08.05.2026).
- [32] Biting Yu u. a. „3D cGAN based cross-modality MR image synthesis for brain tumor segmentation“. en. In: *2018 IEEE 15th International Symposium on Biomedical Imaging (ISBI 2018)*. Washington, DC: IEEE, Apr. 2018, S. 626–630. ISBN: 978-1-5386-3636-7. DOI: 10.1109/ISBI.2018.8363653. URL: <https://ieeexplore.ieee.org/document/8363653/> (besucht am 08.05.2026).
- [33] Dong Nie u. a. „Medical Image Synthesis with Context-Aware Generative Adversarial Networks“. en. In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2017*. Hrsg. von Maxime Descoteaux u. a. Bd. 10435. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2017, S. 417–425. ISBN: 978-3-319-66178-0 978-3-319-66179-7. DOI: 10.1007/978-3-319-66179-7_48. URL: https://link.springer.com/10.1007/978-3-319-66179-7_48 (besucht am 08.05.2026).
- [34] Gihyun Kwon, Chihye Han und Dae-shik Kim. „Generation of 3D Brain MRI Using Auto-Encoding Generative Adversarial Networks“. en. In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2019*. Hrsg. von Dinggang Shen u. a. Bd. 11766. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2019, S. 118–126. ISBN: 978-3-030-32247-2 978-3-030-32248-9. DOI: 10.1007/978-3-030-32248-9_14. URL: https://link.springer.com/10.1007/978-3-030-32248-9_14 (besucht am 08.05.2026).
- [35] Li Sun u. a. „Hierarchical Amortized GAN for 3D High Resolution Medical Image Synthesis“. en. In: *IEEE Journal of Biomedical and Health Informatics* 26.8 (Aug. 2022), S. 3966–3975. ISSN: 2168-2194, 2168-2208. DOI: 10.1109/JBHI.2022.3172976. URL: <https://ieeexplore.ieee.org/document/9770375/> (besucht am 08.05.2026).
- [36] Firas Khader u. a. *Medical Diffusion: Denoising Diffusion Probabilistic Models for 3D Medical Image Generation*. en. arXiv:2211.03364 [eess]. Jan. 2023. DOI: 10.

- 48550/arXiv.2211.03364. URL: <http://arxiv.org/abs/2211.03364> (besucht am 19.04.2026).
- [37] Anna Volokitin u. a. „Modelling the Distribution of 3D Brain MRI Using a 2D Slice VAE“. en. In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2020*. Hrsg. von Anne L. Martel u. a. Bd. 12267. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2020, S. 657–666. ISBN: 978-3-030-59727-6 978-3-030-59728-3. DOI: 10.1007/978-3-030-59728-3_64. URL: https://link.springer.com/10.1007/978-3-030-59728-3_64 (besucht am 08.05.2026).
- [38] Zhisheng Xiao, Karsten Kreis und Arash Vahdat. *Tackling the Generative Learning Trilemma with Denoising Diffusion GANs*. en. arXiv:2112.07804 [cs]. Apr. 2022. DOI: 10.48550/arXiv.2112.07804. URL: <http://arxiv.org/abs/2112.07804> (besucht am 08.05.2026).
- [39] Walter H. L. Pinaya u. a. „Brain Imaging Generation with Latent Diffusion Models“. en. In: *Deep Generative Models*. Hrsg. von Anirban Mukhopadhyay u. a. Bd. 13609. Series Title: Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, 2022, S. 117–126. ISBN: 978-3-031-18575-5 978-3-031-18576-2. DOI: 10.1007/978-3-031-18576-2_12. URL: https://link.springer.com/10.1007/978-3-031-18576-2_12 (besucht am 08.05.2026).
- [40] Zolnamar Dorjsembe u. a. „Conditional Diffusion Models for Semantic 3D Brain MRI Synthesis“. en. In: *IEEE Journal of Biomedical and Health Informatics* 28.7 (Juli 2024). arXiv:2305.18453 [eess], S. 4084–4093. ISSN: 2168-2194, 2168-2208. DOI: 10.1109/JBHI.2024.3385504. URL: <http://arxiv.org/abs/2305.18453> (besucht am 08.05.2026).
- [41] Ian J Goodfellow u. a. „Generative Adversarial Nets“. en. In: ().
- [42] Diederik P. Kingma und Max Welling. *Auto-Encoding Variational Bayes*. en. arXiv:1312.6114 [stat.ML]. Dez. 2022. DOI: 10.48550/arXiv.1312.6114. URL: <http://arxiv.org/abs/1312.6114> (besucht am 14.05.2026).
- [43] Takeshi Okadome. *Essentials of Generative AI*. en. Singapore: Springer Nature Singapore, 2025. ISBN: 978-981-96-0028-1 978-981-96-0029-8. DOI: 10.1007/978-

- 981 - 96 - 0029 - 8. URL: <https://link.springer.com/10.1007/978-981-96-0029-8> (besucht am 15.02.2026).
- [44] Jonathan Ho, Ajay Jain und Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. en. arXiv:2006.11239 [cs]. Dez. 2020. DOI: 10.48550/arXiv.2006.11239. URL: <http://arxiv.org/abs/2006.11239> (besucht am 15.02.2026).
- [45] Alex Nichol und Prafulla Dhariwal. *Improved Denoising Diffusion Probabilistic Models*. en. arXiv:2102.09672 [cs]. Feb. 2021. DOI: 10.48550/arXiv.2102.09672. URL: <http://arxiv.org/abs/2102.09672> (besucht am 17.03.2026).
- [46] Jiaming Song, Chenlin Meng und Stefano Ermon. *Denoising Diffusion Implicit Models*. en. arXiv:2010.02502 [cs]. Okt. 2022. DOI: 10.48550/arXiv.2010.02502. URL: <http://arxiv.org/abs/2010.02502> (besucht am 19.03.2026).
- [47] Cheng Lu u. a. „DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models“. en. In: *Machine Intelligence Research* 22.4 (Aug. 2025). arXiv:2211.01095 [cs], S. 730–751. ISSN: 2731-538X, 2731-5398. DOI: 10.1007/s11633-025-1562-4. URL: <http://arxiv.org/abs/2211.01095> (besucht am 04.05.2026).
- [48] Wenliang Zhao u. a. „UniPC: A Unified Predictor-Corrector Framework for Fast Sampling of Diffusion Models“. en. In: ().
- [49] Marvin John Ignacio u. a. „Revisiting U-Net: a foundational backbone for modern generative AI“. en. In: *Artificial Intelligence Review* 59.2 (Nov. 2025), S. 45. ISSN: 1573-7462. DOI: 10.1007/s10462-025-11450-0. URL: <https://link.springer.com/10.1007/s10462-025-11450-0> (besucht am 24.02.2026).
- [50] Keiron O’Shea und Ryan Nash. *An Introduction to Convolutional Neural Networks*. en. arXiv:1511.08458 [cs]. Dez. 2015. DOI: 10.48550/arXiv.1511.08458. URL: <http://arxiv.org/abs/1511.08458> (besucht am 04.03.2026).
- [51] Nhat-Duc Hoang und Van-Duc Tran. „Deep Learning-Based Predictive Modeling of Urban Heat Stress Using Transformer Network, Deep Neural Network, and Convolutional Neural Network“. en. In: *Remote Sensing in Earth Systems Sciences* 8.4 (Dez. 2025), S. 1161–1184. ISSN: 2520-8195, 2520-8209. DOI: 10.1007/s41976-025-00242-3. URL: <https://link.springer.com/10.1007/s41976-025-00242-3> (besucht am 04.03.2026).
- [52] Wei Shen u. a. *Hands-On Computer Vision*. en. Singapore: Springer Nature Singapore, 2026. ISBN: 9789819548347 9789819548354. DOI: 10.1007/978-981-95-4835-4. URL: <https://link.springer.com/10.1007/978-981-95-4835-4> (besucht am 06.03.2026).
- [53] Alex Krizhevsky, Ilya Sutskever und Geoffrey E. Hinton. „ImageNet classification with deep convolutional neural networks“. en. In: *Communications of the ACM* 60.6 (Mai 2017), S. 84–90. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3065386. URL: <https://dl.acm.org/doi/10.1145/3065386> (besucht am 04.03.2026).
- [54] Olaf Ronneberger, Philipp Fischer und Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. en. arXiv:1505.04597 [cs]. Mai 2015. DOI:

- 10.48550/arXiv.1505.04597. URL: <http://arxiv.org/abs/1505.04597> (besucht am 22.02.2026).
- [55] Nicholas Metropolis und S. Ulam. „The Monte Carlo Method“. In: *Journal of the American Statistical Association* 44.247 (1949). PMID: 18139350, S. 335–341. DOI: 10.1080/01621459.1949.10483310. eprint: <https://www.tandfonline.com/doi/pdf/10.1080/01621459.1949.10483310>. URL: <https://www.tandfonline.com/doi/abs/10.1080/01621459.1949.10483310>.
- [56] James Jordon u. a. *Synthetic Data - what, why and how?* 2022. arXiv: 2205.03257 [cs.LG]. URL: <https://arxiv.org/abs/2205.03257>.
- [57] Mauro Giuffrè^{1/2} und Dennis L. Shung. „Harnessing the power of synthetic data in healthcare: innovation, application, and privacy“. en. In: *npj Digital Medicine* 6.1 (Okt. 2023), S. 186. ISSN: 2398-6352. DOI: 10.1038/s41746-023-00927-3. URL: <https://www.nature.com/articles/s41746-023-00927-3> (besucht am 09.04.2026).
- [58] Bronwyn Loong u. a. „Disclosure control using partially synthetic data for large scale health surveys, with applications to CanCORS“. en. In: *Statistics in Medicine* 32.24 (Okt. 2013), S. 4139–4161. ISSN: 0277-6715, 1097-0258. DOI: 10.1002/sim.5841. URL: <https://onlinelibrary.wiley.com/doi/10.1002/sim.5841> (besucht am 09.04.2026).
- [59] T E Raghunathan, J P Reiter und D B Rubin. „Multiple Imputation for Statistical Disclosure Limitation“. en. In: ().
- [60] *Art. 9 DSGVO – Verarbeitung besonderer Kategorien personenbezogener Daten*. 2018. URL: <https://dsgvo-gesetz.de/art-9-dsgvo/> (besucht am 18.04.2026).
- [61] *Art. 6 KI-VO – Einstufungsvorschriften für Hochrisiko-KI-Systeme*. 2024. URL: <https://artificialintelligenceact.eu/de/article/6/> (besucht am 18.04.2026).
- [62] *Art. 10 KI-VO – Daten und Daten-Governance*. 2024. URL: <https://artificialintelligenceact.eu/de/article/10/> (besucht am 18.04.2026).
- [63] Maja Nisevic, Dusko Milojevic und Daniela Spajic. „Synthetic data in medicine: Legal and ethical considerations for patient profiling“. en. In: *Computational and Structural Biotechnology Journal* 28 (Jan. 2025), S. 190–198. ISSN: 2001-0370. DOI: 10.1016/j.csbj.2025.05.026. URL: <https://spj.science.org/doi/10.1016/j.csbj.2025.05.026> (besucht am 18.04.2026).
- [64] Shanshan Wang u. a. „Generative Artificial Intelligence in Medical Imaging: Foundations, Progress, and Clinical Translation“. en. In: *Research* 8 (Jan. 2025), S. 1029. ISSN: 2639-5274. DOI: 10.34133/research.1029. URL: <https://spj.science.org/doi/10.34133/research.1029> (besucht am 19.04.2026).
- [65] Fei Wang und Anita Preininger. „AI in Health: State of the Art, Challenges, and Future Directions“. en. In: *Yearbook of Medical Informatics* 28.01 (Aug. 2019), S. 016–026. ISSN: 0943-4747, 2364-0502. DOI: 10.1055/s-0039-1677908. URL: [http :](http://)

- //www.thieme-connect.de/DOI/DOI?10.1055/s-0039-1677908 (besucht am 19.04.2026).
- [66] Shenghuan Sun u. a. *Aligning Synthetic Medical Images with Clinical Knowledge using Human Feedback*. en. arXiv:2306.12438 [eess]. Juni 2023. DOI: 10.48550/arXiv.2306.12438. URL: <http://arxiv.org/abs/2306.12438> (besucht am 19.04.2026).
 - [67] Gregory Schuit, Denis Parra und Cecilia Besa. *Perceptual Evaluation of GANs and Diffusion Models for Generating X-rays*. en. arXiv:2508.07128 [cs]. Aug. 2025. DOI: 10.48550/arXiv.2508.07128. URL: <http://arxiv.org/abs/2508.07128> (besucht am 19.04.2026).
 - [68] Pinar Civicioglu und Erkan Besdok. „Image discrimination robustness of classical image quality metrics: an analysis on MAE, MSE, ERGAS, LMSE, SSIM, SAM, UIQI, PSNR, NIQE, PIQE, SNR, VIF, FSIM, GMSD, and NCC“. en. In: *Quality & Quantity* (Okt. 2025). ISSN: 0033-5177, 1573-7845. DOI: 10.1007/s11135-025-02404-3. URL: <https://link.springer.com/10.1007/s11135-025-02404-3> (besucht am 01.03.2026).
 - [69] Martin Heusel u. a. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. en. arXiv:1706.08500 [cs]. Jan. 2018. DOI: 10.48550/arXiv.1706.08500. URL: <http://arxiv.org/abs/1706.08500> (besucht am 01.03.2026).
 - [70] Christian Szegedy u. a. *Rethinking the Inception Architecture for Computer Vision*. en. arXiv:1512.00567 [cs]. Dez. 2015. DOI: 10.48550/arXiv.1512.00567. URL: <http://arxiv.org/abs/1512.00567> (besucht am 01.03.2026).
 - [71] Peng Long und Xiaozhou Guo. *Generative Adversarial Network: Principle and Practice*. en. Singapore: Springer Nature Singapore, 2026. ISBN: 978-981-96-9403-7 978-981-96-9404-4. DOI: 10.1007/978-981-96-9404-4. URL: <https://link.springer.com/10.1007/978-981-96-9404-4> (besucht am 01.03.2026).
 - [72] D. C. Dowson und B. V. Landau. „The Fréchet Distance between Multivariate Normal Distributions“. In: *Journal of Multivariate Analysis* 12.3 (1982), S. 450–455. ISSN: 0047-259X. DOI: 10.1016/0047-259X(82)90077-X.
 - [73] Sho Maruyama. „Properties of the SSIM metric in medical image assessment: correspondence between measurements and the spatial frequency spectrum“. en. In: *Physical and Engineering Sciences in Medicine* 46.3 (Sep. 2023), S. 1131–1141. ISSN: 2662-4729, 2662-4737. DOI: 10.1007/s13246-023-01280-1. URL: <https://link.springer.com/10.1007/s13246-023-01280-1> (besucht am 02.03.2026).
 - [74] Zhou Wang u. a. „Image quality assessment: from error visibility to structural similarity“. en. In: *IEEE Transactions on Image Processing* 13.4 (Apr. 2004), S. 600–

612. ISSN: 1057-7149, 1941-0042. DOI: 10.1109/TIP.2003.819861. URL: <https://ieeexplore.ieee.org/document/1284395/> (besucht am 02.03.2026).
- [75] Jim Nilsson und Tomas Akenine-Möller. *Understanding SSIM*. en. arXiv:2006.13846 [eess]. Juni 2020. DOI: 10.48550/arXiv.2006.13846. URL: <http://arxiv.org/abs/2006.13846> (besucht am 02.03.2026).
- [76] Brian B. Moser u. a. „Diffusion Models, Image Super-Resolution And Everything: A Survey“. en. In: *IEEE Transactions on Neural Networks and Learning Systems* 36.7 (Juli 2025). arXiv:2401.00736 [cs], S. 11793–11813. ISSN: 2162-237X, 2162-2388. DOI: 10.1109/TNNLS.2024.3476671. URL: <http://arxiv.org/abs/2401.00736> (besucht am 23.04.2026).
- [77] Richard Zhang u. a. „The Unreasonable Effectiveness of Deep Features as a Perceptual Metric“. en. In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. Salt Lake City, UT: IEEE, Juni 2018, S. 586–595. ISBN: 978-1-5386-6420-9. DOI: 10.1109/CVPR.2018.00068. URL: <https://ieeexplore.ieee.org/document/8578166/> (besucht am 15.05.2026).
- [78] Lee R. Dice. „Measures of the Amount of Ecologic Association Between Species“. en. In: *Ecology* 26.3 (Juli 1945), S. 297–302. ISSN: 0012-9658, 1939-9170. DOI: 10.2307/1932409. URL: <https://esajournals.onlinelibrary.wiley.com/doi/10.2307/1932409> (besucht am 15.05.2026).
- [79] Kelly H. Zou u. a. „Statistical validation of image segmentation quality based on a spatial overlap index1“. en. In: *Academic Radiology* 11.2 (Feb. 2004), S. 178–189. ISSN: 10766332. DOI: 10.1016/S1076-6332(03)00671-8. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1076633203006718> (besucht am 15.05.2026).
- [80] GeeksforGeeks. *Why is Python the Best-Suited Programming Language for Machine Learning?* <https://www.geeksforgeeks.org/blogs/why-is-python-the-best-suited-programming-language-for-machine-learning/>. Besucht am: 15.03.2026. Aug. 2025.
- [81] Hongwei Bran Li u. a. *The Brain Tumor Segmentation (BraTS) Challenge 2023: Brain MR Image Synthesis for Tumor Segmentation (BraSyn)*. en. arXiv:2305.09011 [eess.IV]. Nov. 2024. DOI: 10.48550/arXiv.2305.09011. URL: <http://arxiv.org/abs/2305.09011> (besucht am 21.05.2026).
- [82] Stefan Elfving, Eiji Uchibe und Kenji Doya. *Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning*. en. arXiv:1702.03118 [cs.LG]. Nov. 2017. DOI: 10.48550/arXiv.1702.03118. URL: <http://arxiv.org/abs/1702.03118> (besucht am 20.05.2026).
- [83] Tim Salimans und Jonathan Ho. *Progressive Distillation for Fast Sampling of Diffusion Models*. en. arXiv:2202.00512 [cs.LG]. Juni 2022. DOI: 10.48550/arXiv.2202.00512. URL: <http://arxiv.org/abs/2202.00512> (besucht am 20.05.2026).
- [84] Tiankai Hang u. a. „Efficient Diffusion Training via Min-SNR Weighting Strategy“. en. In: *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. Paris, France: IEEE, Okt. 2023, S. 7407–7417. ISBN: 979-8-3503-0718-4. DOI: 10.1109/

- ICCV51070 . 2023 . 00684. URL: <https://ieeexplore.ieee.org/document/10378151/> (besucht am 20.05.2026).
- [85] DocCheck Flexikon. *TNM-Klassifikation*. URL: <https://flexikon.doccheck.com/de/TNM-Klassifikation> (besucht am 02.04.2026).
- [86] Cheng Lu u. a. *DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps*. en. arXiv:2206.00927 [cs]. Okt. 2022. DOI: 10.48550/arXiv.2206.00927. URL: <http://arxiv.org/abs/2206.00927> (besucht am 04.05.2026).

A Anhang A

Weitere Abbildungen